

AUTOMATED DETECTION, TRACKING AND TACTICAL ANALYSIS IN BROADCAST FOOTBALL VIDEO

A REPORT SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF BACHELOR OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

Aryan Tiwari

Supervised by Dr Aphrodite Galata

Department of Computer Science

Contents

Abstract	10
Declaration	11
Copyright	12
Acknowledgements	13
1 Introduction	14
1.1 Motivation and Context	14
1.2 Aims and Objectives	15
1.3 Evaluation Strategy	17
1.4 Report Structure	17
2 Background and Theory	18
2.1 Broadcast Football Video Analysis	18
2.2 Related Work	19
2.2.1 Commercial Tracking Systems	19
2.2.2 Academic Benchmarks and Integrated Pipelines	19
2.2.3 Positioning of This Project	20
2.3 Machine Learning and Computer Vision Foundations	20
2.3.1 Supervised Learning for Visual Recognition	20
2.3.2 Neural Networks	21
2.3.3 Convolutional Neural Networks	22
2.3.4 Transfer Learning and Fine-Tuning	23
2.4 Object Detection in Broadcast Football	23
2.4.1 Bounding Boxes, IoU, and Non-Maximum Suppression	24
2.4.2 Precision, Recall, and Mean Average Precision	25

2.4.3	The YOLO Architecture	26
2.5	Multi-Object Tracking	28
2.5.1	Tracking-by-Detection	28
2.5.2	Motion Prediction and Data Association	29
2.5.3	ByteTrack	30
2.5.4	BoT-SORT	30
2.5.5	Tracking Evaluation: HOTA	32
2.6	Team Identification from Visual Features	32
2.6.1	Team Identification as Clustering	32
2.6.2	Visual Representations	33
2.6.3	Dimensionality Reduction	34
2.6.4	k -Means Clustering	34
2.7	Pitch Registration and Planar Homography	35
2.7.1	Planar Homography	35
2.7.2	Estimating H from Point Correspondences	36
2.7.3	Pitch Keypoint Detection	37
2.7.4	Reprojection Error	37
2.8	Spatial and Tactical Analysis in Pitch Space	38
2.8.1	Pitch-Space Trajectories	38
2.8.2	Heatmap Construction	38
2.8.3	Speed and Distance Estimation	39
2.8.4	Possession via Nearest-Player Assignment	39
2.8.5	Offside Rule Formalisation	40
2.9	Chapter Summary	41
3	Methodology and Implementation	42
3.1	System Overview	42
3.2	Requirements and Design Objectives	44
3.3	Development Methodology	46
3.4	Object and Ball Detection	46
3.4.1	Generic Four-Class Detector	47
3.4.2	Dedicated Ball Detector	47
3.5	Multi-Object Tracking	50
3.6	Team Classification	50
3.6.1	Warmup-Phase Integration	51
3.6.2	Torso Cropping	53

3.6.3	Track-Level Temporal Smoothing	53
3.6.4	Goalkeeper Resolution	53
3.7	Pitch Registration	54
3.7.1	Keypoint Detector	55
3.7.2	Stride-Cached Homography Estimation	56
3.8	Tactical Reasoning	57
3.8.1	Pitch-Space Projection	58
3.8.2	Possession Tracking	58
3.8.3	Pass Detection	58
3.8.4	Offside Detection	59
3.8.5	Speed and Distance	60
3.8.6	Heatmaps	61
3.9	Implementation Testing and Verification	61
3.10	System Engineering and Reproducibility	62
3.10.1	Pipeline Orchestration	62
3.10.2	Output Formats	62
3.10.3	Reproducibility	64
3.11	Design Decisions and Failure Handling	66
4	Evaluation	69
4.1	Evaluation Methodology	69
4.2	Detection Performance	70
4.2.1	Generic four-class detector	70
4.2.2	Dedicated ball detector	71
4.3	Tracking Performance	72
4.4	Team Classification Performance	74
4.5	Pitch Registration Accuracy	76
4.6	Tactical Output Quality	78
4.7	End-to-End Performance	80
4.8	Error Budget and Failure Mode Analysis	81
4.9	Discussion	81
5	Conclusion	84
5.1	Summary of Achievements	84
5.2	Critical Reflection	85
5.3	Future Work	86

5.4	Closing Statement	87
-----	-----------------------------	----

Word Count: 14950

List of Tables

3.1	Intermediate representations between pipeline components.	44
3.2	Functional and non-functional requirements.	45
3.3	Ball-class performance: generic vs. dedicated detector.	49
3.4	Component-level verification performed during development.	63
3.5	Training hyperparameters for the three fine-tuned models.	65
3.6	Principal architectural decisions and trade-offs.	67
3.7	Failure conditions and mitigation strategies.	68
4.1	Generic detector: per-class test performance.	70
4.2	Ball-class detection: generic versus dedicated detector.	71
4.3	Tracker comparison on the combined evaluation set.	73
4.4	Team classifier: full-crop clustering and warm-up-split results across three clips.	75
4.5	Pitch-space reprojection error in centimetres, with and without kp_16 ablation.	77
4.6	Pass detection: per-clip and aggregate precision/recall.	79
4.7	Per-clip tactical-output summary.	79
4.8	Per-stage runtime on Apple M2 Pro for a 750-frame clip.	81
4.9	Error budget across pipeline stages.	82
4.10	Summary of pre-registered success criteria against achieved results. . .	83

List of Figures

2.1	A common CNN architecture.	22
2.2	Intersection over Union (IoU).	24
2.3	Illustration of Non-Maximum Suppression.	25
2.4	Generalised YOLO-style detection pipeline.	27
2.5	ByteTrack two-stage association process.	31
3.1	Pipeline architecture overview.	43
3.2	Representative output of the generic four-class detector.	48
3.3	Temporal coherence filtering for the dedicated ball detector.	49
3.4	Tracking continuity through close player interaction.	51
3.5	Warmup-phase diagnostic for team assignment.	52
3.6	Image-space goalkeeper resolution.	54
3.7	Pitch-keypoint template used for registration.	55
3.8	Pitch registration in action.	56
3.9	Offside decision in image space and pitch space.	60
3.10	Per-track heatmap construction.	61
4.1	Generic detector: precision–recall curves.	71
4.2	Generic detector: normalised confusion matrix.	72
4.3	Dedicated ball detector: precision–recall curve.	73
4.4	Example torso crops from clip 1.	75
4.5	Per-keypoint mean pixel error on the test split.	77

List of Abbreviations

AP	Average Precision
AssA	Association Accuracy
CIoU	Complete Intersection over Union
CLIP	Contrastive Language–Image Pre-training
CNN	Convolutional Neural Network
COCO	Common Objects in Context
DetA	Detection Accuracy
DINOv2	self-Distillation with NO labels (version 2)
DLT	Direct Linear Transform
F1	F1 Score (harmonic mean of precision and recall)
FN	False Negative
FP	False Positive
HOTA	Higher Order Tracking Accuracy
HSV	Hue, Saturation, Value
IDF1	Identification F1
IFAB	International Football Association Board
INT8	8-bit Integer (quantisation format)
IoU	Intersection over Union
JSON	JavaScript Object Notation
Lab	CIE L*a*b* Colour Space
MAE	Mean Absolute Error
mAP	mean Average Precision
MOT	Multi-Object Tracking
MOTA	Multi-Object Tracking Accuracy
MP4	MPEG-4 Part 14
MSE	Mean Squared Error
NMS	Non-Maximum Suppression

NPY	NumPy Array File Format
PCA	Principal Component Analysis
PNG	Portable Network Graphics
RANSAC	Random Sample Consensus
R-CNN	Region-based Convolutional Neural Network
ReLU	Rectified Linear Unit
RGB	Red, Green, Blue
SigLIP	Sigmoid Loss for Language–Image Pre-training
SORT	Simple Online and Realtime Tracking
SSD	Single Shot MultiBox Detector
TP	True Positive
UMAP	Uniform Manifold Approximation and Projection
YOLO	You Only Look Once

Abstract

Positional and tracking data are central to modern football analytics, but remain largely confined to elite settings through proprietary commercial systems. Broadcast video offers a more accessible and reproducible source of tactical information, yet open pipelines commonly stop at intermediate outputs such as detections, tracks, or top-down visualisations rather than the event-level analysis that motivates tracking data.

This project develops and evaluates a modular computer vision pipeline that converts broadcast football video into tactical outputs: bird’s-eye visualisations, heatmaps, possession assignments, pass and offside event logs, and per-player match statistics. The system combines a YOLOv8 four-class detector with a dedicated ball detector, ByteTrack multi-object tracking, unsupervised SigLIP-UMAP- k -Means team classification, and a YOLOv8-pose pitch-keypoint detector feeding a stride-cached Direct Linear Transform (DLT) homography estimator. All downstream reasoning is performed in pitch-space coordinates.

All six pre-registered success criteria are met within the defined evaluation setting. The generic detector achieves test $\text{mAP}_{50} = 0.855$, while the dedicated ball detector improves ball-class mAP_{50} from 0.459 to 0.950. ByteTrack attains $\text{HOTA} = 0.668$ on a combined 3,250-frame evaluation set and outperforms BoT-SORT on HOTA, IDF1, and identity-switch count. Team classification reaches mean macro- $F_1 = 0.993$ across three independent broadcast clips. At tactical level, pass detection achieves $F_1 = 0.903$, all ten flagged offside-position events were manually verified as correct, and pitch-space reprojection error remains within the tactical-scale tolerance, with a median error of 169.9 cm and a mean error of 199.4 cm.

The main contribution is an end-to-end, reproducible broadcast-analysis pipeline that requires no per-match manual labelling at inference time and produces quantitatively evaluated and visually inspectable tactical outputs. The results show that broadcast-only football analysis can move beyond detection and tracking towards interpretable tactical summaries, while identifying key limitations in homography accuracy, possession transitions, and speed estimation.

Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

- i. The author of this report (including any appendices and/or schedules to this report) owns certain copyright or related rights in it (the “Copyright”) and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.
- ii. Copies of this report, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.
- iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the “Intellectual Property”) and any reproductions of copyright works in the report, for example graphs and tables (“Reproductions”), which may be described in this report, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.
- iv. Further information on the conditions under which disclosure, publication and commercialisation of this report, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see <http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420>), in any relevant Thesis restriction declarations deposited in the University Library, The University Library’s regulations (see <http://www.library.manchester.ac.uk/about/regulations/>) and in The University’s policy on presentation of Theses

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Aphrodite Galata, for her invaluable guidance, support, and advice throughout this project.

I am also grateful to Kunj Dhola and Selim Sayed for their collaboration and contributions to the project.

I acknowledge the use of generative AI tools to improve the written English, spelling, and grammar of text that I had drafted myself, in line with the project's permitted use of AI. The technical content, methodology, system design, experimental work, analysis, and conclusions presented in this report are my own. A full declaration of AI usage is provided in the project's AI declaration form submitted alongside this report.

Finally, I would like to thank my family and friends for their continued encouragement and support throughout my studies.

Chapter 1

Introduction

1.1 Motivation and Context

Data and analytics have become central to professional football, with clubs and national federations adopting analytical methods for performance analysis and decision-making [28]. Positional and tracking data are particularly valuable, providing richer descriptions of player movement, team organisation, and tactical behaviour than traditional notational analysis [14]. Reliable spatial match data is therefore increasingly important.

Despite this, high-quality football data remain difficult to access outside elite settings. Advanced event and tracking data are often proprietary, expensive, or dependent on specialist collection systems, which limits their availability for public and reproducible work [11, 7]. This motivates the investigation of alternative ways to generate structured football data from more widely available sources.

Broadcast video is an attractive candidate because it is widely available and contains the spatial information needed for tactical analysis. However, recovering structured data from a single moving camera under realistic broadcast conditions is technically demanding. This project therefore treats broadcast football analysis as a pipeline of specialised stages: detection, tracking, team identification, pitch registration, and tactical reasoning. Modern broadcast pipelines typically build on single-stage convolutional detectors and tracking-by-detection frameworks, an approach this work also adopts; transformer-based alternatives exist but are not yet standard in this regime [34]. The interactions between these stages determine the reliability of the final outputs [46, 42].

Open benchmark efforts such as SoccerNet-Tracking and SoccerNet Game State

Reconstruction demonstrate that broadcast football video can support tracking and game-state reconstruction in realistic settings [5, 42]. However, these systems typically produce intermediate outputs such as tracked player positions or minimap reconstructions rather than directly usable tactical outputs. What remains missing is a publicly reproducible system that goes beyond reconstruction to produce *tactical* outputs, including possession, passes, offsides, and per-player statistics, directly from broadcast video, without per-match manual labelling.

1.2 Aims and Objectives

To address the gap identified in Section 1.1, the project investigates how far a modular computer vision pipeline can extract reliable tactical and spatial information from broadcast football video without specialised capture systems or manual team labels, complementing rather than replacing commercial tracking systems.

To achieve this aim, the project pursues the following objectives:

- develop and evaluate a detection approach for the main on-pitch entity classes, including players, goalkeepers, referees, and the ball, from broadcast football footage;
- assess whether multi-object tracking can preserve sufficiently consistent identities over time to support downstream tactical analysis;
- test whether unsupervised clustering of visual embeddings can separate players by team without manual team labels;
- measure the geometric accuracy of pitch keypoint detection and homography estimation for projecting image-space positions into pitch-space;
- determine whether the resulting projected trajectories can support higher-level tactical outputs, including bird’s-eye visualisations, heatmaps, possession estimates, pass events, and offside-related analysis; and
- evaluate the pipeline as an end-to-end system, including how errors in detection, tracking, and homography estimation affect downstream analytical outputs.

The project will be considered successful if the following pre-registered quantitative and qualitative evaluation criteria are met:

- (i) the main object detector achieves a mean Average Precision $\text{mAP}_{50} \geq 0.80$ across entity classes on a held-out test split;
- (ii) multi-object tracking reaches a Higher Order Tracking Accuracy $\text{HOTA} \geq 0.45$ on a labelled evaluation set spanning at least one public benchmark and one in-domain broadcast clip;
- (iii) unsupervised team assignment achieves a macro- $F_1 \geq 0.85$ on held-out broadcast clips under a deployment-realistic warm-up split;
- (iv) pitch registration attains a median pitch-space reprojection error ≤ 250 cm, sufficient to support tactical-scale spatial reasoning;
- (v) pass detection achieves $F_1 \geq 0.80$ against visually-verified ground truth on independent broadcast clips; and
- (vi) the integrated pipeline produces bird’s-eye projections, per-track heatmaps, and offside flags that are qualitatively consistent with the source footage across all evaluation clips.

These thresholds are non-trivial but achievable, informed by published baselines on comparable broadcast tasks. Theiner et al. [46] report player-detection AP values of approximately 0.87–0.93 on broadcast football, with their CenterTrack-based detector achieving roughly 0.90 AP, and report median end-to-end player-position errors on the order of 1–3 m depending on dataset and evaluation setting; FootAndBall [19] similarly reports player AP in the 0.87–0.93 range. The chosen mAP_{50} threshold of 0.80 therefore sits below the reported baseline range, requiring competent rather than state-of-the-art detection. On the SoccerNet-Tracking benchmark [5], baseline trackers without ground-truth detections achieve HOTA scores of approximately 0.37–0.58, so the HOTA threshold of 0.45 sits within this range as a non-trivial but achievable target. Earlier team-classification work used RGB colour histograms with unsupervised clustering [43]; the 0.85 team-assignment threshold is set conservatively to retain only high-confidence assignments. The pass-detection threshold reflects the absence of a directly comparable public benchmark and is set at the level above which event detection becomes analytically useful rather than indicative.

1.3 Evaluation Strategy

The pipeline is evaluated at both *component* and *system* level, because downstream outputs such as possession, heatmaps, and offside decisions depend not only on the performance of individual modules but also on how errors propagate across detection, tracking, and pitch registration stages. This is a position established in recent broadcast-football evaluation work, which recognises component metrics alone as insufficient evidence of integrated system quality [46, 42]. Component evaluation asks whether each stage meets its technical objective on its own task; system evaluation asks whether the integrated pipeline produces tactical outputs that are coherent and useful as a complete system.

At component level, each module is assessed using standard task-appropriate metrics from the computer vision literature: mean Average Precision (mAP) for detection, Higher Order Tracking Accuracy (HOTA) for tracking, F_1 score for team classification, and reprojection error for pitch registration. At system level, the pipeline is judged by the quantitative quality of its tactical outputs (pass-detection F_1 , offside positional correctness) and by qualitative assessment of bird’s-eye and heatmap renderings against the source footage, following the approach used in recent game-state reconstruction benchmark [42]. The detailed annotation procedures, evaluation sets, and per-component methodology are presented in Chapter 4.

1.4 Report Structure

The remainder of this report is organised to guide the reader from the background knowledge needed to understand the system, through its design, to a critical assessment of its performance. Chapter 2 (*Background and Theory*) reviews the relevant literature and technical foundations, establishing the context from which the pipeline design decisions follow. Chapter 3 (*Methodology and Implementation*) describes the architecture and implementation of each pipeline stage, building directly on the methods and models introduced in the background. Chapter 4 (*Evaluation*) then assesses the project at both component and system level, using the metrics and strategy outlined in Section 1.3, presenting the outputs produced in practice alongside failure analysis and the effect of error propagation on downstream outputs. Finally, Chapter 5 (*Conclusion*) summarises the main achievements, reflects on limitations, and outlines directions for future work.

Chapter 2

Background and Theory

2.1 Broadcast Football Video Analysis

Broadcast football footage poses several coupled technical challenges that distinguish it from analysis based on dedicated tracking infrastructure [46, 26]. Where multi-camera or sensor-based systems can rely on fixed reference frames and direct measurement, a broadcast pipeline must recover all player, ball, and pitch information from a single moving-camera video stream [14].

Four properties of this medium drive the design of any such pipeline. The camera continuously pans, tilts, and zooms, so the background cannot serve as a fixed spatial reference and inter-frame motion cannot be attributed to scene objects alone [46]. The ball is small relative to the frame and is frequently blurred, occluded, or temporarily absent, making its detection substantially harder than that of players [5, 19]. Players on the same team share near-identical appearance, including uniform colour, similar build, so identity must be inferred from subtle spatial and temporal cues [46]. Finally, image-space measurements suffer perspective distortion, so quantities such as distance, speed, and tactical position are meaningful only after registration to a canonical pitch model [45].

These constraints have driven the field toward a modular pipeline [46, 26]: object and ball detection produce per-frame observations; multi-object tracking maintains temporal identity; team assignment recovers affiliation; pitch registration projects coordinates into field space; and higher-level outputs such as heatmaps, possession, off-side are computed in pitch coordinates rather than in the image plane.

Formally, the pipeline is a composition of stages,

$$y = f_n(f_{n-1}(\cdots f_1(x))), \quad (2.1)$$

where x is the input video, each f_k is a processing stage, and y is a final pitch-space output such as a heatmap, possession estimate, or offside decision. This composed structure has an important consequence: every output depends on the quality of every upstream stage, so the pipeline cannot be analysed by considering its components in isolation.

2.2 Related Work

This section reviews existing commercial systems, public benchmarks, and academic pipelines for broadcast football analysis, in order to identify the gap that motivates an open pipeline producing tactical outputs from broadcast video alone.

2.2.1 Commercial Tracking Systems

Professional clubs typically obtain positional data from commercial optical or sensor-based systems. TRACAB uses multi-camera optical tracking and has been shown to attain sub-metre positional accuracy under validated match conditions [25], while providers such as Hudl StatsBomb and Second Spectrum supply event data, tracking data, and tactical analysis tools to clubs and broadcasters [7]. These systems are not, however, a viable foundation for open research: their algorithms, data-collection procedures, and raw outputs are proprietary, which limits reproducibility, fair comparison across methods, and applicability to matches where commercial tracking data are unavailable [11, 7]. This motivates approaches that recover spatial and tactical information directly from broadcast footage.

2.2.2 Academic Benchmarks and Integrated Pipelines

The SoccerNet ecosystem provides the largest open benchmark suite for broadcast football. SoccerNet-v2 introduced annotations for action spotting, camera-shot segmentation, and replay detection [8]; SoccerNet-Tracking added bounding-box annotations and standard evaluation protocols for multi-object tracking [5]; and SoccerNet Game State Reconstruction frames the problem as recovering the full game state on

a top-down minimap, jointly evaluating detection, tracking, identification, and pitch registration [42].

Integrated academic systems address overlapping subsets of this pipeline. Theiner et al. [46] extract positional data by combining detection, tracking, and pitch registration, but evaluate only positional accuracy rather than downstream tactical reasoning. Magera et al. [31] (BroadTrack) target broadcast camera calibration specifically, leaving downstream stages out of scope. Maglo et al. [32] pair field registration with self-supervised re-identification to localise players from a single moving view. Earlier work by Beetz et al. [3] demonstrated the feasibility of automated game analysis but predates modern deep-learning detectors and trackers.

A consistent pattern is that these systems stop at intermediate representations, such as bounding boxes, tracks, registered pitches, or minimap reconstructions. A reconstruction may be visually plausible while still containing localisation, identity, or ball-detection errors that propagate into derived quantities such as possession, pass detection, distance estimates, or offside decisions. Evaluation at the reconstruction level alone therefore does not establish tactical usefulness.

2.2.3 Positioning of This Project

The literature thus leaves a clear gap: closed commercial systems produce tactical outputs but are not reproducible, while open academic systems are reproducible but stop short of tactical interpretation. This project addresses that gap by extending an open broadcast pipeline beyond reconstruction, treating detection, tracking, and pitch registration as inputs to downstream pitch-space statistics rather than as final outputs.

2.3 Machine Learning and Computer Vision Foundations

2.3.1 Supervised Learning for Visual Recognition

In supervised learning, a model learns from labelled examples. A training set

$$\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$$

pairs each input x_i — for instance an image or image crop — with a label y_i , which may be an object class, a bounding box, a set of keypoints, or a combination of these.

The model is a function f_{θ} with learnable parameters θ , and training minimises the empirical risk

$$\theta^* = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \mathcal{L}(f_{\theta}(x_i), y_i),$$

where $\mathcal{L}$ measures the discrepancy between prediction and ground truth. Two losses recur in this project. Cross-entropy is used for classification targets,

$$\mathcal{L}_{\text{CE}}(\hat{y}, y) = - \sum_{c=1}^C y_c \log \hat{y}_c,$$

where $\hat{y}$ is a predicted distribution over C classes and y is a one-hot ground truth. Mean Squared Error (MSE) is used for continuous targets such as bounding-box coordinates,

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|\hat{y}_i - y_i\|_2^2.$$

Optimisation is performed by stochastic gradient descent or one of its variants, with parameters updated as

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L},$$

where η is the learning rate [15]. The supervised components of this project — the object detector and the pitch keypoint detector — all instantiate this framework, learning visual patterns from annotated examples rather than from hand-written rules.

2.3.2 Neural Networks

A neural network is a parametric model composed of layers of simple computational units. Each unit combines its inputs through learned weights and biases and applies a non-linear activation. For layer l ,

$$\mathbf{h}^{(l)} = \sigma \left(W^{(l)} \mathbf{h}^{(l-1)} + \mathbf{b}^{(l)} \right),$$

where $W^{(l)}$ and $\mathbf{b}^{(l)}$ are the layer's learnable parameters and σ is an element-wise non-linearity. The non-linearity is essential: a composition of linear maps is itself linear, so without σ the entire network would collapse to a single linear transformation. The most common choice is the rectified linear unit,

$$\text{ReLU}(z) = \max(0, z),$$

which is computationally cheap and tends to produce sparse activations that aid optimisation [15]. A fully-connected network of this form, however, treats an image as a flat vector, which scales poorly with image size and discards spatial structure. Convolutional architectures, introduced next, address both issues.

2.3.3 Convolutional Neural Networks

Convolutional neural networks (CNNs) are designed for grid-structured data such as images. Rather than connecting every input to every output, a CNN applies small learned filters across local regions of the image, so the same visual pattern — an edge, a texture, a goalpost — can be detected regardless of its position in the frame. A two-dimensional convolutional layer computes

$$y_{i,j,k} = \sigma \left(\sum_{u,v,c} W_{u,v,c,k} x_{i+u,j+v,c} + b_k \right),$$

where x is the input feature map, W is a learned filter, b_k is a bias, and $y_{i,j,k}$ is the output at spatial location (i,j) on output channel k ; the indices u,v range over the filter's spatial extent and c over input channels. Weight sharing across spatial positions makes CNNs both more parameter-efficient than fully-connected networks and translation-equivariant by construction. Figure 2.1 illustrates this structure.

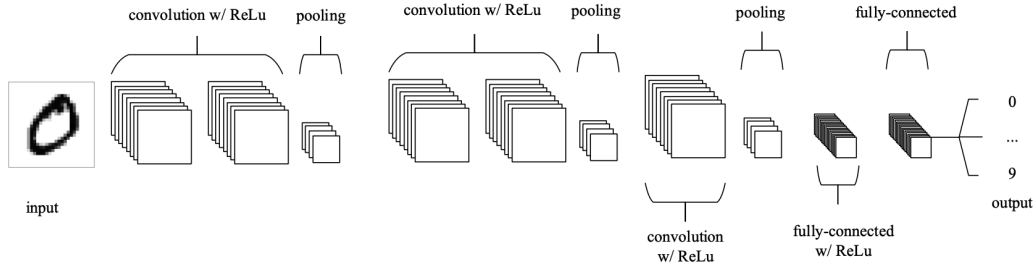


Figure 2.1: A common CNN architecture: stacked convolution, activation, and pooling stages produce feature maps of decreasing spatial resolution, which are passed to fully connected layers for prediction. Reproduced from O’Shea and Nash [36].

Pooling layers reduce spatial resolution and enlarge the effective receptive field of subsequent filters. Max pooling, the most common variant, takes

$$y_{i,j} = \max_{(u,v) \in \mathcal{R}_{i,j}} x_{u,v}$$

over a local region $\mathcal{R}_{i,j}$. Stacking convolution, activation, and pooling yields a hierarchical representation: early layers respond to local patterns such as edges and colour gradients, while deeper layers compose these into semantic features such as body parts, kit textures, or pitch markings. This compositional behaviour was already present in early systems such as LeNet [22], and deep CNNs became dominant in computer vision after their breakthrough on large-scale image classification [20]. In this project, CNN backbones provide the visual features for both object detection and pitch keypoint localisation; the detection-specific architecture of YOLO is discussed in Section 2.4.

2.3.4 Transfer Learning and Fine-Tuning

Training deep visual models from scratch typically requires very large datasets and substantial computation. Transfer learning sidesteps this by initialising the target-task model with parameters θ_0 learned from a large general-purpose dataset such as ImageNet or Common Objects in Context (COCO). Empirically, the lower layers of such pre-trained networks encode general visual features — edges, colour transitions, textures — that transfer well across domains, while upper layers are more task-specific [47]. Fine-tuning continues training from θ_0 on the target task, often with a smaller learning rate so that useful low-level features are preserved while higher-level representations adapt. This regime is well-suited to broadcast football analysis: labelled football data are scarce, but pre-training already supplies robust features that generalise across the camera-angle, scale, lighting, and motion-blur variations characteristic of broadcast video.

2.4 Object Detection in Broadcast Football

Object detection is the task of locating and classifying objects within an image. In broadcast football the relevant objects are players, goalkeepers, referees, and the ball.

Formally, a detector maps an input image I to a set of predictions

$$D = \{(b_i, c_i, s_i)\}_{i=1}^N, \quad (2.2)$$

where b_i is a bounding box, c_i is the predicted class label, and $s_i \in [0, 1]$ is a confidence score reflecting the model’s certainty that an object of class c_i is present at b_i .

2.4.1 Bounding Boxes, IoU, and Non-Maximum Suppression

A bounding box is a rectangular region enclosing an object, parameterised either by corner coordinates or by centre, width, and height. The standard measure of localisation quality between a predicted box B_p and a ground-truth box B_g is the Intersection over Union (IoU), illustrated in Figure 2.2,

$$\text{IoU}(B_p, B_g) = \frac{|B_p \cap B_g|}{|B_p \cup B_g|}, \quad (2.3)$$

which takes values in $[0, 1]$, with 1 indicating perfect overlap. A prediction is counted as a true positive only if its IoU with an unmatched ground-truth box exceeds a chosen threshold and the predicted class is correct [24].

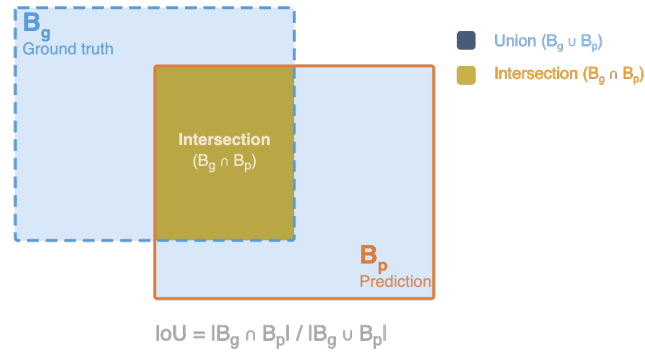


Figure 2.2: Intersection over Union (IoU) between a ground-truth bounding box B_g (blue, dashed) and a predicted bounding box B_p (orange). IoU is the ratio of the intersection area (gold) to the total union area of both boxes, computed as $\text{IoU} = |B_g \cap B_p| / |B_g \cup B_p|$.

Detectors typically emit several overlapping predictions for the same object. Non-Maximum Suppression (NMS) reduces these to a single detection per object via a greedy procedure: given a candidate set $\mathcal{B}$ of scored predictions and an overlap threshold τ_{NMS} , at each step

$$b^* = \arg \max_{b \in \mathcal{B}} s(b),$$

the highest-scoring remaining detection b^* is retained and any $b \in \mathcal{B}$ with $\text{IoU}(b, b^*) \geq \tau_{\text{NMS}}$ is discarded; the procedure repeats until $\mathcal{B}$ is empty [38]. Figure 2.3 illustrates this process. NMS matters in football footage because players cluster during tackles, set-pieces, and crowded midfield phases — without it, a single player can be counted

multiple times and generate duplicate tracks downstream.

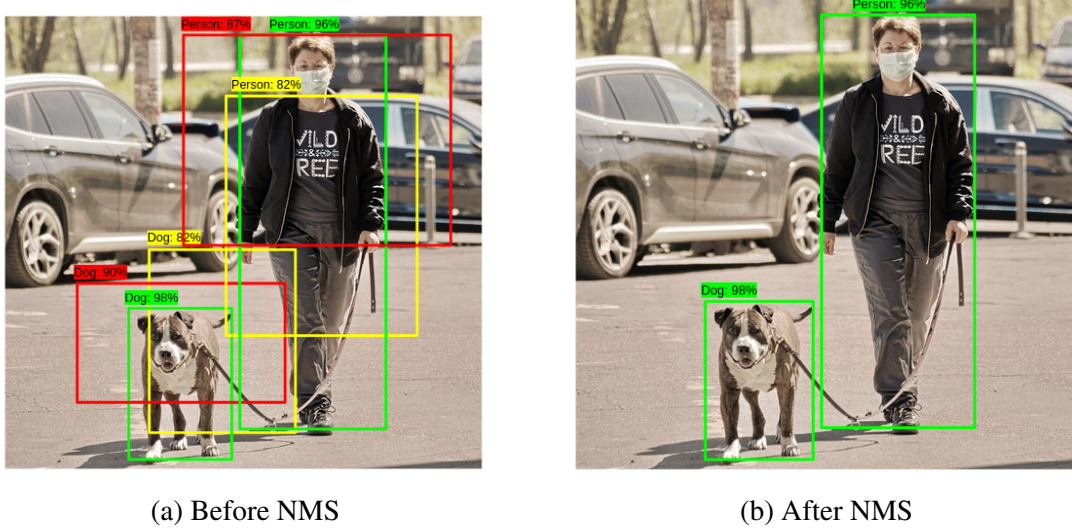


Figure 2.3: Illustration of Non-Maximum Suppression. Before NMS, multiple overlapping candidate boxes may be produced for the same object. After NMS, lower-confidence overlapping boxes are suppressed and the most confident detection is retained.

2.4.2 Precision, Recall, and Mean Average Precision

Once detections have been matched to ground truth, performance is summarised by precision and recall. Precision measures how many predicted detections are correct, while recall measures how many ground-truth objects are successfully detected:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \quad \text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad (2.4)$$

where TP, FP, and FN denote true positives, false positives, and false negatives. Varying the confidence threshold traces out a precision–recall curve: raising the threshold improves precision at the cost of recall, and vice versa.

Average Precision (AP) summarises this curve as the area beneath it,

$$\text{AP} = \int_0^1 p(r) dr,$$

typically approximated by interpolation over a finite set of recall points. Mean Average

Precision (mAP) averages AP across classes,

$$\text{mAP} = \frac{1}{C} \sum_{c=1}^C \text{AP}_c.$$

Two variants matter for this project. mAP_{50} fixes the IoU threshold at 0.50, while mAP_{50-95} averages mAP over IoU thresholds from 0.50 to 0.95 in steps of 0.05 [24] and is therefore a stricter measure of localisation. The distinction is operationally important: a loosely localised player box may suffice for visualisation but not for tracking association or pitch projection.

2.4.3 The YOLO Architecture

Object detectors fall broadly into two families. Two-stage detectors such as Faster Region-based CNN (R-CNN) [40] first propose candidate regions and then classify and refine them, giving strong localisation at the cost of speed. Single-stage detectors such as the Single Shot MultiBox Detector (SSD) [27] and YOLO [38] predict boxes and class scores in a single forward pass, making them better suited to video-rate applications such as broadcast football analysis.

YOLO (*You Only Look Once*) treats detection as a single regression problem. The image is divided into a grid, and each cell is responsible for predicting any object whose centre falls inside it: a bounding box, an *objectness* score reflecting whether an object is present at all, and a distribution over class labels [38]. This one-shot design makes YOLO fast, but its accuracy depends critically on how the network is structured to handle objects of very different sizes — in broadcast football, a near-camera player and a far-touchline ball can differ in pixel area by two orders of magnitude.

Architecture. A modern YOLO-style detector consists of three stages, illustrated in Figure 2.4. A convolutional *backbone* processes the image and produces feature maps at several spatial resolutions: shallower maps preserve fine spatial detail useful for small objects, while deeper maps carry richer semantic information useful for classification. A *neck*, typically based on feature pyramid networks [23], fuses these maps so that every prediction is informed by both fine detail and broader context. The *detection head* reads from the fused maps and outputs, at each spatial location, a bounding box, an objectness score, and class probabilities. A confidence threshold and Non-Maximum Suppression then reduce these per-location predictions to the final

detection set.

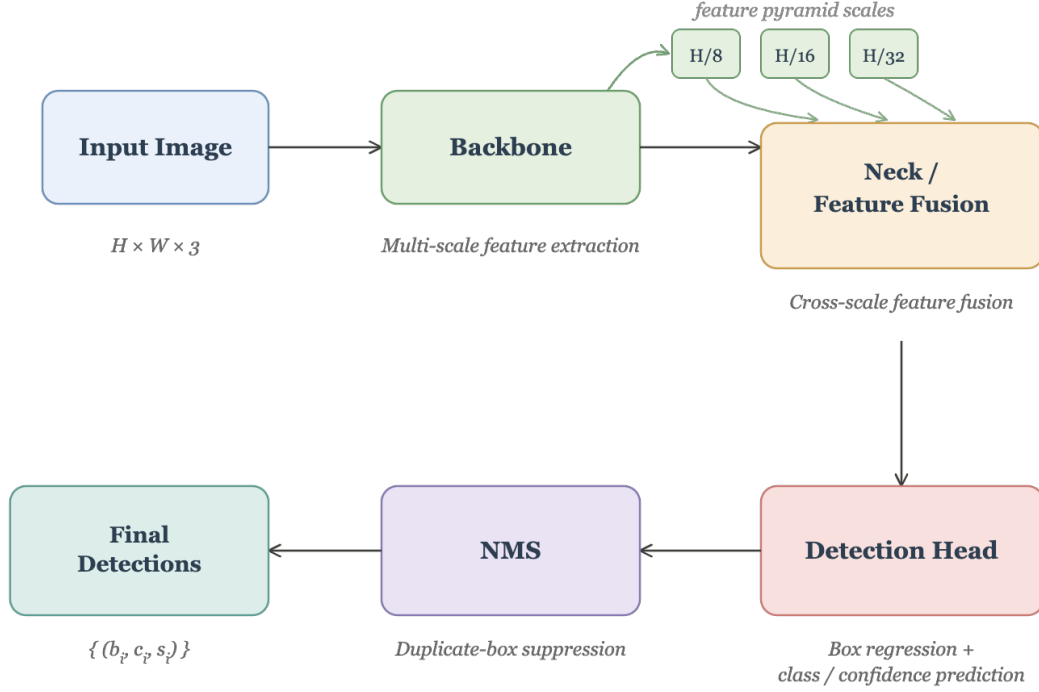


Figure 2.4: Generalised YOLO-style single-stage detection pipeline. The backbone extracts multi-scale feature maps, the neck fuses them, and the detection head predicts boxes, objectness, and class scores. NMS removes duplicate predictions to produce the final detection set $\{(b_i, c_i, s_i)\}$.

How boxes are predicted. Rather than predicting absolute pixel coordinates directly, the detection head predicts box geometry relative to each feature-map location, which is more numerically stable to train. Earlier YOLO variants used anchor boxes, which are hand-defined prior shapes that the network adjusts during prediction [39]. YOLOv8 instead adopts an anchor-free design in which each feature-map location directly regresses the four sides of the bounding box as distances from that location [17]. This removes the need to tune anchor priors and simplifies the detection head.

Training objective. YOLO is trained to minimise a weighted sum of three losses, one for each output of the head:

$$\mathcal{L} = \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{obj}} \mathcal{L}_{\text{obj}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}}.$$

The first term $\mathcal{L}_{\text{box}}$ penalises localisation error, typically using an IoU-based loss such as Complete IoU (CIoU). The second $\mathcal{L}_{\text{obj}}$ is a binary loss over whether each predicted location actually contains an object. The third $\mathcal{L}_{\text{cls}}$ is a classification loss over the object class. Splitting objectness from classification lets the model learn the question *is something here?* independently of *what is it?*, which helps when classes overlap visually — as players and goalkeepers do in broadcast football.

2.5 Multi-Object Tracking

Object detection produces independent bounding boxes in each frame, but does not say whether a player detected in one frame is the same player detected in the next. Multi-object tracking (MOT) addresses this by linking detections over time and assigning a persistent identity to each object [9].

Formally, let

$$D_t = \{d_t^1, \dots, d_t^{n_t}\} \quad (2.5)$$

denote the detections at frame t , each carrying a bounding box, class, and confidence, and let

$$\mathcal{T}_{t-1} = \{\tau_{t-1}^1, \dots, \tau_{t-1}^{m_{t-1}}\} \quad (2.6)$$

denote the active tracks from the previous frame. Tracking must assign each $d \in D_t$ to a track in $\mathcal{T}_{t-1}$, create new tracks for unmatched detections, and gracefully handle tracks for which no matching detection arrives.

2.5.1 Tracking-by-Detection

Most modern MOT systems follow the *tracking-by-detection* paradigm [4]: an object detector is applied independently to each frame, and a separate tracker associates the resulting detections over time. This decouples visual recognition from temporal association — the detector answers *what is in this frame?* while the tracker answers *which detections belong to the same object?* The framework is modular (detector improvements can be dropped in without redesigning the tracker) and online (associations are

produced frame-by-frame, without waiting for the full sequence). Its principal weakness is detector dependence: missed detections fragment tracks, false positives spawn spurious tracks, and inaccurate boxes produce incorrect associations — failures that cluster in crowded scenes such as set-pieces and midfield contests.

2.5.2 Motion Prediction and Data Association

A standard motion model is the Kalman filter, which recursively estimates the state of a linear-Gaussian system from noisy observations [18]. For a tracked bounding box, a common state under a constant-velocity assumption is

$$\mathbf{x}_t = [x_t, y_t, w_t, h_t, \dot{x}_t, \dot{y}_t, \dot{w}_t, \dot{h}_t]^\top,$$

combining the box centre, its width and height, and their rates of change. The filter alternates between two steps. The *predict* step propagates the previous state ($\mathbf{x}_{t-1}$) and its covariance (P_{t-1}) forward,

$$\hat{\mathbf{x}}_t^- = F\mathbf{x}_{t-1}, \quad P_t^- = FP_{t-1}F^\top + Q, \quad (2.7)$$

where F is the state-transition matrix encoding the motion model and Q is the process-noise covariance. The predicted state $\hat{\mathbf{x}}_t^-$ gives the expected location of the track before seeing the detector output for frame t . The *update* step then corrects this prediction using a matched detection $\mathbf{z}_t$:

$$K_t = P_t^- H^\top (HP_t^- H^\top + R)^{-1}, \quad (2.8)$$

$$\mathbf{x}_t = \hat{\mathbf{x}}_t^- + K_t(\mathbf{z}_t - H\hat{\mathbf{x}}_t^-), \quad P_t = (I - K_t H)P_t^-, \quad (2.9)$$

where H is the observation matrix, R is the measurement-noise covariance, and the Kalman gain K_t weighs the new evidence against the prior prediction, trusting the detection more when measurement noise is small relative to predicted state uncertainty. The term $\mathbf{z}_t - H\hat{\mathbf{x}}_t^-$ is the difference between the observed detection and the predicted measurement.

Given predicted track locations, association reduces to a bipartite matching problem in which each track receives at most one detection and vice versa. We construct a cost matrix $C \in \mathbb{R}^{m \times n}$ with

$$C_{ij} = 1 - \text{IoU}(\hat{B}_i, B_j), \quad (2.10)$$

where $\hat{B}_i$ is the predicted box for track i and B_j is the box of detection j ; lower cost means better match. The Hungarian algorithm [21] solves this assignment problem to optimality in $O(n^3)$ time [10]. Unmatched detections seed new tracks; unmatched tracks are buffered and eventually deleted if not recovered. This Kalman–Hungarian skeleton underlies Simple Online and Realtime Tracking (SORT) [4], ByteTrack [49], and BoT-SORT [2].

2.5.3 ByteTrack

ByteTrack [49] observes that low-confidence detections are not all false: confidence frequently drops during occlusion, motion blur, or truncation while the object remains present. Discarding them therefore loses useful evidence. ByteTrack instead partitions detections by confidence,

$$D_t^{\text{high}} = \{d \in D_t : s_d \geq \tau_{\text{high}}\}, \quad D_t^{\text{low}} = \{d \in D_t : s_d < \tau_{\text{high}}\}, \quad (2.11)$$

where s_d is the confidence score of detection d , τ_{high} is the high-confidence threshold, and runs association in two passes (Figure 2.5). Tracks are first matched against D_t^{high} using the Hungarian algorithm with IoU cost; any track that remains unmatched is then given a second chance against D_t^{low} . High-confidence detections that fail to match seed new tracks; tracks unmatched after both passes are held in a buffer for a short period before deletion. This design preserves track continuity through short occlusions and detector dropouts while still anchoring associations to high-confidence evidence.

2.5.4 BoT-SORT

BoT-SORT [2] extends the tracking-by-detection skeleton with two refinements aimed at challenging video. First, *camera-motion compensation*: when the camera pans, tilts, or zooms, all bounding-box positions shift together, and naïve IoU between predicted and observed boxes becomes unreliable. BoT-SORT estimates a global affine transform between consecutive frames — typically by matching sparse keypoints in the static background — and pre-warps each predicted track location into the current frame’s coordinates before association. Second, *appearance-based re-identification*: visual features extracted from detection crops disambiguate spatially similar candidates, particularly during occlusions. Both refinements add per-frame computational cost relative to ByteTrack.

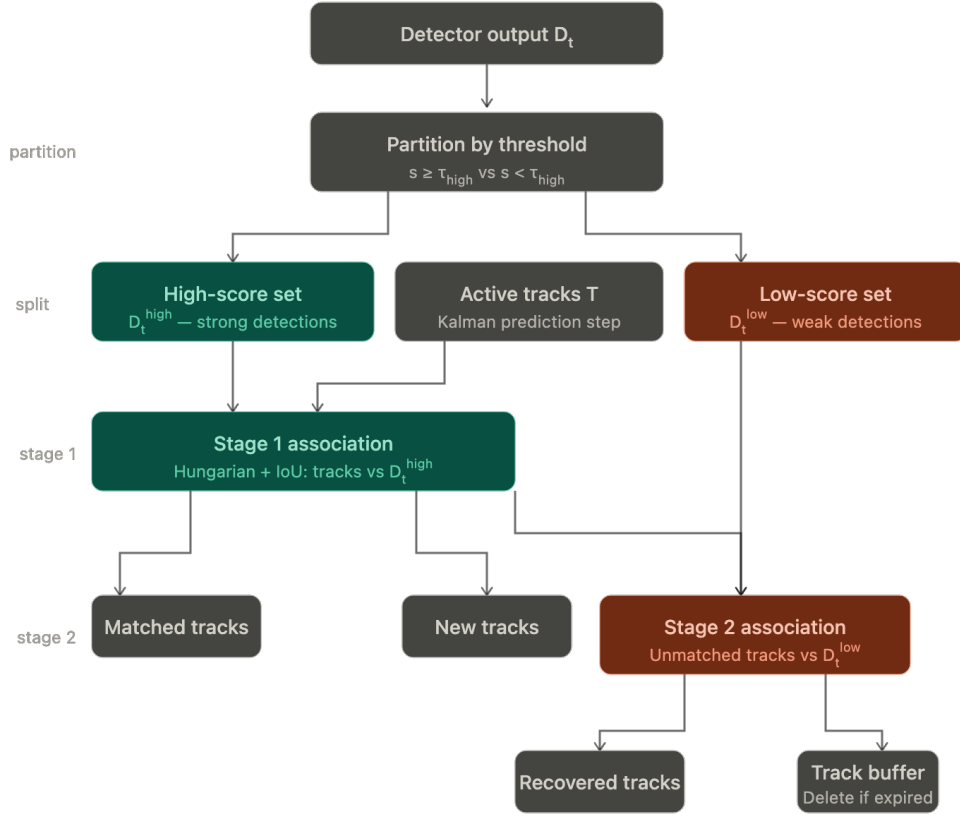


Figure 2.5: ByteTrack two-stage association process. Detector output is partitioned by confidence threshold τ_{high} into a high-score set D_t^{high} and a low-score set D_t^{low} . Stage 1 matches active tracks against D_t^{high} using the Hungarian algorithm with IoU cost; unmatched tracks proceed to Stage 2, where they are compared against D_t^{low} . Tracks that remain unmatched after both stages are held in a buffer and deleted if unrecovered within a fixed number of frames.

2.5.5 Tracking Evaluation: HOTA

Tracking quality must capture two properties simultaneously: how accurately objects are localised, and how consistently their identities persist over time. Multi-Object Tracking Accuracy (MOTA) and Identification F1 (IDF1) are established baselines [9], but both conflate detection and association in ways that obscure the source of errors.

Higher Order Tracking Accuracy (HOTA) addresses this by decomposing performance into separate detection and association scores [29]:

$$\text{HOTA}_\alpha = \sqrt{\text{DetA}_\alpha \cdot \text{AssA}_\alpha}, \quad (2.12)$$

where the Detection Accuracy DetA_α measures box overlap with ground truth at localisation threshold α , and the Association Accuracy AssA_α measures identity consistency — the average proportion of ground-truth and predicted trajectories that share an identity over their joint lifetime. The geometric mean ensures that neither component can dominate: a tracker that localises well but fragments identities, or associates well but misses many objects, will be penalised in both factors. The final HOTA score averages Equation 2.12 over $\alpha \in [0.05, 0.95]$, giving a single threshold-independent summary.

2.6 Team Identification from Visual Features

Object detection and tracking determine where players are and how they move, but neither recovers which team each player belongs to. Because players on the same team share a visually similar kit, the task is naturally cast as an unsupervised grouping problem on player appearance [43].

2.6.1 Team Identification as Clustering

Each detected player crop I_i is mapped to a feature vector

$$\mathbf{z}_i = f(I_i), \quad \mathbf{z}_i \in \mathbb{R}^d, \quad (2.13)$$

by a feature extractor f chosen so that crops of players in the same kit produce similar vectors and crops of opposing players produce dissimilar ones. The team-identification problem is then to partition $\{\mathbf{z}_i\}_{i=1}^N$ into groups corresponding to the teams on the pitch — a standard unsupervised clustering problem [30]. For the two outfield teams the

natural choice is two clusters, since their kits are required by competition rules to be visually distinguishable.

2.6.2 Visual Representations

The classical approach extracts colour histograms in spaces such as Red-Green-Blue (RGB), Hue-Saturation-Value (HSV), or CIE L*a*b* (Lab) and clusters players by histogram similarity [43]. Such features are fast, interpretable, and align with the cue humans use, but they are sensitive to lighting changes, shadows, and background pixels included in the crop, and they discard spatial structure entirely.

A more robust alternative is to use deep visual embeddings. Modern pretrained encoders such as Contrastive Language-Image Pre-training (CLIP), Sigmoid Loss for Language-Image Pre-training (SigLIP), and self-Distillation with NO labels v2 (DI-NOv2) learn transferable visual representations from internet-scale pre-training [37, 48, 35] and produce embeddings that capture colour, texture, shape, and pattern jointly in a single vector. Because their training data covers extensive visual variation, the resulting features generalise to new domains — including broadcast football crops — without any domain-specific fine-tuning, which makes them well-suited to a setting in which match-specific labels are unavailable.

For clustering, embeddings ($\mathbf{z}_i$) are typically L2-normalised so that

$$\tilde{\mathbf{z}}_i = \mathbf{z}_i / \|\mathbf{z}_i\|_2,$$

and similarity is measured by cosine similarity,

$$\text{sim}(\mathbf{z}_i, \mathbf{z}_j) = \frac{\mathbf{z}_i^\top \mathbf{z}_j}{\|\mathbf{z}_i\|_2 \|\mathbf{z}_j\|_2}.$$

On the unit sphere, cosine similarity and squared Euclidean distance differ only by a constant — $\|\tilde{\mathbf{z}}_i - \tilde{\mathbf{z}}_j\|_2^2 = 2 - 2\tilde{\mathbf{z}}_i^\top \tilde{\mathbf{z}}_j$ — so Euclidean-distance clustering on normalised embeddings is equivalent to clustering by cosine similarity. This makes pretrained embeddings directly usable with the distance-based clustering algorithms below.

2.6.3 Dimensionality Reduction

Deep embeddings are typically high-dimensional ($d \in [512, 1024]$ for the encoders cited above), and distance-based clustering becomes less stable in such spaces because pairwise distances concentrate — many points appear nearly equidistant, a phenomenon known as the curse of dimensionality [1]. Dimensionality reduction maps embeddings into a lower-dimensional space $\mathbb{R}^{d'}$ with $d' \ll d$ before clustering:

$$\phi : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}, \quad \mathbf{y}_i = \phi(\mathbf{z}_i). \quad (2.14)$$

The classical linear choice is Principal Component Analysis (PCA), but its linearity cannot unfold the curved manifold structure that deep embedding models often produce. UMAP (Uniform Manifold Approximation and Projection) is a non-linear alternative that constructs a weighted neighbourhood graph in the high-dimensional space and optimises a low-dimensional layout to preserve its local topology [33]. The objective minimises the cross-entropy between the high- and low-dimensional neighbourhood distributions, which preserves the local similarity structure that matters for team identification: players in the same kit should remain close, even when their global arrangement on a manifold is non-linear.

2.6.4 k -Means Clustering

Given reduced features $\{\mathbf{y}_i\}_{i=1}^N$, k -means partitions them into k clusters by minimising the within-cluster sum of squared distances [30]:

$$\min_{\{\mu_j\}_{j=1}^k} \sum_{i=1}^N \min_{1 \leq j \leq k} \|\mathbf{y}_i - \mu_j\|_2^2, \quad (2.15)$$

where μ_j is the centroid of cluster j . This objective is non-convex, so direct minimisation is intractable. In practice it is solved by Lloyd’s algorithm, which alternates two steps until convergence. Given current centroids $\{\mu_j\}$, each point is assigned to its nearest centroid,

$$a_i = \arg \min_{1 \leq j \leq k} \|\mathbf{y}_i - \mu_j\|_2^2,$$

and each centroid is then updated as the mean of the points assigned to it,

$$\mu_j = \frac{1}{|\{i : a_i = j\}|} \sum_{i: a_i = j} \mathbf{y}_i.$$

Each step strictly decreases or preserves the objective, guaranteeing convergence to a local minimum. The local optimum reached depends on initialisation; *k*-means++ provides a probabilistic seeding scheme that selects initial centroids far apart in feature space, giving provably better expected performance than uniform random initialisation. For team identification on outfield players $k = 2$, and the algorithm recovers the two groups up to an arbitrary label permutation — separating *which player is on which team*, but not *which cluster corresponds to which named team*.

2.7 Pitch Registration and Planar Homography

Detections and tracks are produced in image coordinates, but tactical analysis — bird’s-eye visualisation, heatmaps, distance and speed estimation, possession analysis, offside reasoning — requires positions in a common pitch coordinate system. Pitch registration estimates a geometric mapping between the broadcast image and a canonical top-down pitch model, allowing image-space observations to be projected into pitch space [46, 45].

2.7.1 Planar Homography

The football pitch is approximately planar, and projective geometry tells us that any two views of a plane are related by a 3×3 projective transformation called a *planar homography* [16]. Working in homogeneous coordinates, an image point $\tilde{\mathbf{x}} = (u, v, 1)^\top$ and its corresponding pitch-space point $\tilde{\mathbf{X}} = (x, y, 1)^\top$ are related by

$$\tilde{\mathbf{X}} \sim H\tilde{\mathbf{x}}, \quad (2.16)$$

where $\sim$ denotes equality up to a non-zero scale factor. Because H is defined only up to scale, it has eight degrees of freedom rather than nine. Writing

$$H = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{pmatrix},$$

the explicit pitch coordinates of an image point are

$$x = \frac{h_{11}u + h_{12}v + h_{13}}{h_{31}u + h_{32}v + h_{33}}, \quad y = \frac{h_{21}u + h_{22}v + h_{23}}{h_{31}u + h_{32}v + h_{33}}, \quad (2.17)$$

which makes the projective nature explicit: the denominator is what produces perspective foreshortening, and reduces to an affine transformation only when $h_{31} = h_{32} = 0$.

The mapping is valid only for points lying on (or very near) the pitch plane. Player feet are therefore the correct reference point for projection, and the bottom centre of each player's bounding box serves as a practical approximation to the foot location.

2.7.2 Estimating H from Point Correspondences

Estimating H requires correspondences $\{(\mathbf{x}_i, \mathbf{X}_i)\}_{i=1}^N$ between image and pitch points. Each correspondence yields *two* independent constraints on H : from Equation 2.16, the cross product $\tilde{\mathbf{X}}_i \times H\tilde{\mathbf{x}}_i = \mathbf{0}$ gives three scalar equations, of which only two are linearly independent. Since H has eight degrees of freedom, at least four non-collinear correspondences are required to determine it; with more than four, the system becomes overdetermined.

The standard solution is the Direct Linear Transform (DLT) algorithm [16]. Stacking the two constraints per correspondence produces a homogeneous linear system

$$A\mathbf{h} = \mathbf{0}, \quad A \in \mathbb{R}^{2N \times 9}, \quad (2.18)$$

where $\mathbf{h} \in \mathbb{R}^9$ is the vectorised form of H . The least-squares solution under the constraint $\|\mathbf{h}\|_2 = 1$ is the right singular vector of A corresponding to its smallest singular value, obtained from the singular value decomposition of A . In practice, image and pitch coordinates are first normalised — typically translated to have zero mean and scaled to have unit average distance from the origin — to improve numerical conditioning [16] before solving.

Real correspondences contain both noise and outliers — incorrectly localised key-points whose error is large enough to corrupt the least-squares estimate. Robust estimators such as Random Sample Consensus (RANSAC) [13] mitigate this by fitting candidate homographies on minimal correspondence sets and retaining the one with the largest inlier consensus, tolerating outlier rates well above 50% — the standard choice when correspondences come from an automatic detector rather than manual annotation.

2.7.3 Pitch Keypoint Detection

The correspondences required by DLT are obtained by detecting semantic landmarks on the playing surface — line intersections, penalty-area corners, centre-circle tangencies, and similar distinctive points formed by pitch markings — whose pitch-space coordinates are fixed by the laws of the game and therefore known a priori. Detecting these landmarks in the broadcast image yields exactly the $(\mathbf{x}_i, \mathbf{X}_i)$ pairs that Equation 2.18 requires.

This is structurally a keypoint detection problem and is solved by the same family of CNN architectures used for human pose estimation, with the target points replaced by fixed pitch landmarks. A convolutional backbone extracts multi-scale feature maps and a prediction head outputs per-landmark confidences and locations [17]. Keypoint quantity, accuracy, and spatial spread all matter: more correspondences overdetermine the system and average out localisation noise; correspondences distributed across the visible pitch constrain the transformation more tightly than correspondences clustered in one region.

2.7.4 Reprojection Error

Registration quality is measured by reprojection error. For correspondence i ,

$$e_i = \|\mathbf{X}_i - \pi(H\tilde{\mathbf{x}}_i)\|_2, \quad (2.19)$$

where $\pi(\cdot)$ converts homogeneous coordinates back to Cartesian by dividing by the third component. The mean reprojection error over N correspondences,

$$\bar{e} = \frac{1}{N} \sum_{i=1}^N e_i, \quad (2.20)$$

summarises how closely the estimated H maps the detected landmarks back to their known pitch locations.

Low reprojection error is necessary but not sufficient for accurate downstream projection. The metric is computed only on the keypoints used to fit H , so a homography that fits its inputs perfectly can still extrapolate poorly to regions of the pitch where no landmarks were detected — particularly when keypoints cluster in one half of the image, leaving the other half geometrically underconstrained.

2.8 Spatial and Tactical Analysis in Pitch Space

Once detections and tracks have been mapped to a canonical pitch model, analysis can proceed in a coordinate system that is independent of the broadcast camera. This matters because pixel distances do not correspond uniformly to real-world distances under projective viewing: equal displacements in image space can encode very different real displacements depending on where they occur in the frame. Pitch-space coordinates remove this distortion, making it possible to compare positions, compute movement statistics, and reason about tactical events in a common geometric frame [46, 14].

2.8.1 Pitch-Space Trajectories

Each tracked player must be reduced to a single image point before the homography is applied. Any such point must lie on the ground plane, since the homography is valid only there; an image point above the plane projects incorrectly because its line of sight does not intersect the pitch where the player stands.

Let $\mathbf{q}_t^{(i)} \in \mathbb{R}^2$ be the chosen ground-plane image point for player i at frame t . Given an image-to-pitch homography H , the projected pitch-space position is

$$\mathbf{p}_t^{(i)} = \pi\left(H\tilde{\mathbf{q}}_t^{(i)}\right), \quad (2.21)$$

where $\tilde{\mathbf{q}}_t^{(i)}$ is the homogeneous form of the image point and $\pi(\cdot)$ converts back to Cartesian coordinates. The sequence $\{\mathbf{p}_t^{(i)}\}_{t=1}^T$ is the pitch-space trajectory of player i , and is the geometric primitive on which all subsequent spatial analysis is built.

2.8.2 Heatmap Construction

A heatmap summarises the spatial distribution of a player's or team's positions over time. The pitch is divided into a regular grid, and each projected position contributes to the cell containing it:

$$H_i(u, v) = \sum_{t=1}^T \mathbf{1}\left[(u, v) = \text{bin}\left(\mathbf{p}_t^{(i)}\right)\right], \quad (2.22)$$

where $\mathbf{1}[\cdot]$ is the indicator function and $\text{bin}(\cdot)$ maps a continuous pitch coordinate to a discrete grid cell. Equation 2.22 is a two-dimensional histogram of the trajectory, and a team-level heatmap is obtained by accumulating positions from all players assigned

to that team.

Histogram estimates are noisy, particularly when sample counts per cell are small. Smoothing with an isotropic Gaussian kernel,

$$\tilde{H}_i = H_i * G_\sigma, \quad (2.23)$$

yields a kernel density estimate of the underlying spatial distribution. The bandwidth σ controls the bias–variance trade-off: a larger σ suppresses noise but blurs spatial detail; a smaller σ preserves detail at the cost of higher variance per cell.

2.8.3 Speed and Distance Estimation

Movement statistics derive from temporal differences of the trajectory. With a video frame rate f_s and inter-frame interval $\Delta t = 1/f_s$, for player i , a finite-difference velocity estimate at frame t is

$$\mathbf{v}_t^{(i)} = \frac{\mathbf{p}_t^{(i)} - \mathbf{p}_{t-1}^{(i)}}{\Delta t}, \quad (2.24)$$

where $\mathbf{p}_t^{(i)}$ and $\mathbf{p}_{t-1}^{(i)}$ are consecutive pitch-space positions. The corresponding instantaneous speed is

$$s_t^{(i)} = \|\mathbf{v}_t^{(i)}\|_2. \quad (2.25)$$

Total distance covered can then be approximated by summing the Euclidean displacement between consecutive positions:

$$d^{(i)} = \sum_{t=2}^T \left\| \mathbf{p}_t^{(i)} - \mathbf{p}_{t-1}^{(i)} \right\|_2. \quad (2.26)$$

Finite differencing is intrinsically noise-amplifying: small per-frame localisation errors become disproportionately large per-frame velocity errors. This is a property of the differencing operator itself rather than of any particular pipeline.

2.8.4 Possession via Nearest-Player Assignment

Possession and passes are not detected directly from a frame. They are inferred from the spatial relationship between the ball, the players, and their team identities over time. The simplest model assigns the ball to the spatially closest eligible player. Given a ball position $\mathbf{b}_t \in \mathbb{R}^2$ and a candidate set $\mathcal{A}_t$ of players' pitch positions $\mathbf{p}_t^{(i)}$, the assigned

holder is

$$h_t = \arg \min_{i \in \mathcal{A}_t} \|\mathbf{b}_t - \mathbf{p}_t^{(i)}\|_2. \quad (2.27)$$

This is exactly the assignment of $\mathbf{b}_t$ to a cell of the Voronoi diagram of $\{\mathbf{p}_t^{(i)}\}$, in which each player owns the region of pitch closer to them than to any other player. A pass corresponds to a transition of h_t between distinct players, normally on the same team.

More refined formulations such as *pitch control* [12] weight each player’s influence over the pitch by position and velocity, capturing dynamic context such as momentum, reach, time-to-arrive that nearest-player assignment ignores. This expressiveness comes at the cost of requiring reliable per-player velocities and orientations, which in turn demand higher upstream localisation quality than the nearest-player rule.

2.8.5 Offside Rule Formalisation

Offside is fundamentally a spatial rule and therefore admits a pitch-space formalisation. Under Law 11 of the Laws of the Game, a player is in an offside position if, at the moment the ball is played by a team-mate, they are in the opponents’ half and nearer to the opponents’ goal line than both the ball and the second-last opponent [44].

Choose a coordinate system in which the attacking direction of the team in possession is the positive x -axis. Let $x_t^{(a)}$, $x_t^{(b)}$, and $x_t^{(o_2)}$ be the longitudinal coordinates at the moment of the pass of an attacking player, the ball, and the second-last opponent respectively. The positional offside test is then

$$x_t^{(a)} > \max(x_t^{(b)}, x_t^{(o_2)}), \quad (2.28)$$

combined with the requirement that the attacker is in the opponents’ half. The maximum expresses the rule’s “beyond both” condition: the second-last opponent serves as the defensive line, but the ball overrides this when it is ahead of that line.

Equation 2.28 captures positional offside only. Law 11 additionally requires the player to become actively involved in play before being penalised, which is a behavioural rather than purely spatial condition [44]; this falls outside what pitch-space geometry alone can decide. The formalisation nonetheless illustrates how a tactical rule reduces to a geometric inequality once players, ball, and pitch share a coordinate system.

2.9 Chapter Summary

This chapter has developed the theoretical foundation for the broadcast football analysis pipeline. The unifying view is that the system is a composition of stages — detection, tracking, team identification, pitch registration, and pitch-space tactical reasoning — each formalised mathematically and each consuming the outputs of those before it. This structure makes the pipeline modular and individually testable, but also tightly coupled: errors at any stage propagate downstream, so upstream quality bounds what later stages can achieve. The methods, metrics, and notation introduced here are applied directly in the implementation described in Chapter 3 and in the evaluation presented in Chapter 4.

Chapter 3

Methodology and Implementation

This chapter describes the design and implementation of the pipeline introduced in Chapter 1, building on the theory of Chapter 2. Sections mirror the data flow: each corresponds to one processing stage f_k of the composed pipeline $y = f_n(\dots f_1(x))$ (Equation 2.1), reusing the same notation. Empirical evaluation against the success criteria of Section 1.2 is deferred to Chapter 4.

3.1 System Overview

The pipeline ingests a broadcast football video and emits four outputs: an annotated MP4 video with bounding boxes, team labels, track identifiers, and offside indicators; a bird’s-eye-view MP4 in pitch-space coordinates; per-track heatmap PNG images; and JSON logs of pass and offside events with per-player match statistics. Processing decomposes into three parallel branches that converge at a shared pitch-space representation (Figure 3.1).

Three design decisions distinguish this architecture from a single-detector baseline. First, the ball-class outputs of the generic detector are discarded in favour of a dedicated single-class ball model, motivated by per-class imbalance and small-object difficulty (Section 3.4). Second, goalkeeper team identity is resolved by a spatial heuristic rather than the unsupervised clusterer because competition rules require goalkeeper kits to be visually distinct from both outfield kits, so a two-cluster model cannot place them reliably (Section 3.6). Third, pitch keypoint inference is the slowest pipeline stage; it runs every fifteen frames and the resulting homography is cached, exploiting broadcast football’s slow camera motion (Section 3.7).

At the convergence point, each player’s bottom-centre image-space anchor and the

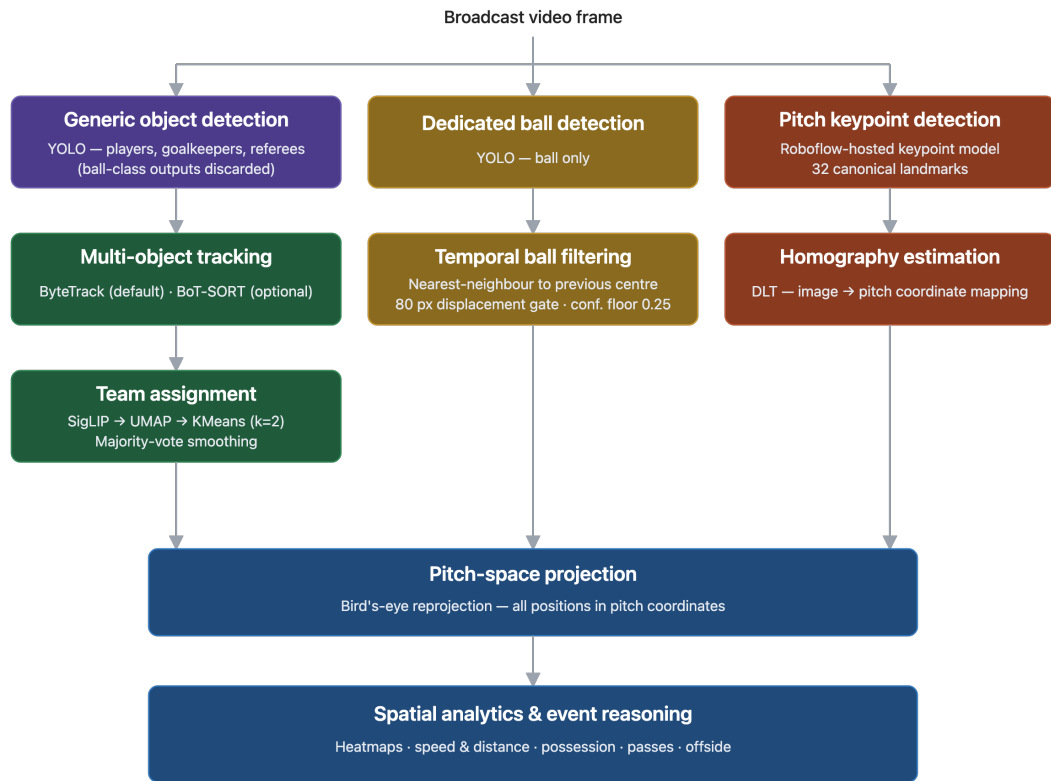


Figure 3.1: Pipeline architecture. Three parallel branches process each frame: a generic object detector with multi-object tracker producing per-frame player and referee detections that are then assigned to teams (left); a dedicated ball detector with a temporal coherence filter (centre); and a pitch keypoint detector whose outputs are used to estimate a per-frame homography by Direct Linear Transform (right). The three branches converge at a shared pitch-space representation, on which all spatial analytics — heatmaps, speed and distance, possession, passes, and offside — are computed.

ball’s position are mapped through the cached homography, following the ground-plane validity argument of Section 2.8.1. All downstream analytics operate on these pitch-space positions (Section 3.8).

A separate *warmup phase* runs once at the start of each video, fitting the unsupervised SigLIP-UMAP- k -Means clusterer of Section 2.6.4 on player crops sampled from the first twenty seconds of footage. Although outside the per-frame loop, it operates on the same input, so the pipeline produces team labels without per-match labelling effort, directly addressing project Objective 3 (Section 1.2).

Table 3.1 records the intermediate representations passed between modules. The remainder of the chapter follows the order of Figure 3.1.

#	Representation	Contents	Produced by	Consumed by
1	Raw detection	bbox, class, confidence	Generic YOLO detector (§3.4)	Tracker; ball filter
2	Track	bbox, class, conf, track ID	ByteTrack/BoT-SORT (§3.5)	Team classifier; projection
3	Filtered ball detection	bbox, confidence	Dedicated ball detector + temporal filter (§3.4)	Possession tracker
4	Team-labelled track	track ID, raw label, smoothed label	TeamAssigner (§3.6)	Possession; offside; renderer
5	Pitch keypoints	image-space coords + per-keypoint confidence	YOLO-pose model (§3.7)	Homography estimator
6	Homography	3×3 image-to-pitch transform	DLT estimator (§3.7)	ViewTransformer
7	Pitch-space position	(x, y) in centimetres on the pitch	ViewTransformer (§3.8)	Heatmaps; speed; possession; offside
8	Event log	passes, offsides, per-track statistics	Tactical reasoning modules (§3.8)	JSON outputs; Ch. 4

Table 3.1: Intermediate representations passed between pipeline components, with the section defining each producer indicated in parentheses.

3.2 Requirements and Design Objectives

The aims and objectives of Chapter 1 commit the system to six functional capabilities and a set of pre-registered quantitative success criteria. Table 3.2 records these as

functional (FR) and non-functional (NFR) requirements, with explicit traceability to the originating objectives and to the implementation sections that deliver them; subsequent sections return to the FR/NFR identifiers when describing how each requirement is met.

ID	Requirement	Source	Implemented by
<i>Functional</i>			
FR1	Detect players, goalkeepers, referees, and the ball with $\text{mAP}_{50} \geq 0.80$ across entity classes.	Obj. 1; SC (i)	Generic four-class detector and dedicated ball detector (Sec. 3.4).
FR2	Maintain player and referee identities across frames with $\text{HOTA} \geq 0.45$ on a labelled evaluation set.	Obj. 2; SC (ii)	Multi-object tracker with selectable algorithm (Sec. 3.5).
FR3	Assign tracked players to teams without per-match manual labels, achieving $\text{macro-}F_1 \geq 0.85$ under a deployment-realistic warm-up split.	Obj. 3; SC (iii)	SigLIP-UMAP- k -Means clustering with track-level smoothing and goalkeeper resolver (Sec. 3.6).
FR4	Project detections into pitch-space with median reprojection error ≤ 250 cm for tactical-scale spatial reasoning.	Obj. 4; SC (iv)	YOLO-pose pitch-keypoint detector and DLT homography estimator (Sec. 3.7).
FR5	Produce possession, pass and offside events, speed and distance statistics, and heatmaps, with pass detection $F_1 \geq 0.80$.	Obj. 5; SC (v), (vi)	Tactical reasoning modules operating in pitch-space (Sec. 3.8).
FR6	Operate end-to-end, producing bird's-eye projections, heatmaps, and offside flags consistent with the source footage.	Obj. 6; SC (vi)	Two-phase pipeline orchestration with cached intermediates (Sec. 3.10).
<i>Non-functional</i>			
NFR1	Components are independently replaceable without modifying unrelated modules.	Obj. 6	Package layout under <code>src/</code> and configuration-file tracker selection (Sec. 3.10).
NFR2	The system is reproducible from documented weights, datasets, hyperparameters, and configuration files.	Obj. 6	Reproducibility documentation (Sec. 3.10.3).
NFR3	Outputs support human inspection and machine-readable downstream analysis.	Obj. 5	MP4 and PNG visualisations plus NPY and JSON exports (Sec. 3.10).

Table 3.2: Functional (FR) and non-functional (NFR) requirements, their source objectives and success criteria, and the implementation sections that address them.

3.3 Development Methodology

Development was iterative and component-wise: each processing stage in Figure 3.1 was implemented and inspected in isolation against the explicit input/output contracts of Table 3.1 before integration into the end-to-end loop. Tactical outputs depend on the cumulative correctness of detection, tracking, team assignment, and pitch registration, so under-tested components fail at the analytics stage in ways difficult to attribute to their source; isolating each module kept this failure surface small and deferred integration until each had been independently verified.

Component design decisions were informed first by the literature of Chapter 2 and then checked empirically on held-out development clips. The architecture-level choices — the two-detector split, the spatial goalkeeper resolver, the cached homography — emerged from this loop: the literature established the available techniques and their known failure modes (Sections 2.4, 2.6, and 2.7), while inspection of intermediate outputs exposed the specific failure modes motivating each design departure. The empirical justifications throughout this chapter (most prominently the per-class detection breakdown of Section 3.4) are diagnostic evidence from this loop, distinct from the quantitative evaluation of Chapter 4.

Integration parameters — temporal smoothing windows, confidence thresholds, displacement gates for plausibility checks — were calibrated on the same clips to suppress transient errors without rejecting plausible behaviour. They were not selected by formal hyperparameter search: the project’s contribution is the surrounding architecture and component-level decisions, not fine-grained tuning. Where a parameter is principled (for example, a sprinter-ceiling speed cap or a frame-rate-derived smoothing window) the rationale is stated alongside the value at its introduction. Implementation-level testing is documented in Section 3.9.

3.4 Object and Ball Detection

The pipeline uses two YOLO detectors. A four-class *generic detector* locates players, goalkeepers, referees, and the ball, and a single-class *dedicated ball detector* re-runs ball localisation on every frame. Only the player, goalkeeper, and referee predictions of the generic detector flow downstream; its ball-class outputs are discarded for the empirical reasons developed below. Together these models address FR1 and Objective 1 (Section 1.2).

3.4.1 Generic Four-Class Detector

The generic detector is a YOLOv8m model [17] fine-tuned on the *Football_object_detection_dataset* (Roboflow), comprising 768 broadcast frames split 618/75/75 across train/validation/test. Of 18,838 instances across the four classes, players account for 15,297 — $12.6\times$ the ball class (1,213) and $26.3\times$ the goalkeeper class (581). This imbalance has direct consequences for per-class performance.

The medium variant (yolov8m.pt) was selected as it balances accuracy and inference cost. The pipeline targets commodity hardware without a discrete GPU, so a larger backbone would dominate end-to-end runtime. Inputs were resized to 1280×1280 to preserve small-object detail for ball localisation, constraining batch size to 4 on the NVIDIA A100. Training ran for 50 epochs using Ultralytics 8.4.21 on PyTorch 2.8.0 with the framework’s default augmentation regime, completing in approximately 15 minutes. Full hyperparameters are catalogued in Section 3.10.3.

Held-out test mAP_{50} reached 0.855, clearing the 0.80 threshold of SC (i); full per-class metrics appear in Chapter 4. One result motivates the section that follows: the Player, Goalkeeper, and Referee classes each reach mAP_{50} above 0.98, but the Ball class achieves only 0.459 with recall 0.393. The normalised confusion matrix shows 31% of true balls misclassified as background and 40% of Ball predictions being background false positives. This recall collapse on the ball — alongside near-perfect detection of the three large-object classes — is the diagnostic evidence motivating a separate ball detector.

Figure 3.2 shows a representative frame annotated by the generic four-class detector.

3.4.2 Dedicated Ball Detector

Two structural factors drive the recall collapse. First, imbalance: the ball appears once per image while players appear roughly twenty times, so localisation loss on player boxes dominates the gradient signal. Second, scale: at broadcast distance the ball occupies only a handful of pixels, far smaller than the typical anchor distribution of single-stage detectors [38, 39]. A single-class detector trained on a ball-curated dataset removes the imbalance entirely and lets model capacity focus on scale.

The dedicated detector is a YOLOv8x model trained on the *ball_dataset* (Roboflow), comprising 1,165 frames split 932/114/119, each with exactly one ball annotation. The xlarge variant is acceptable here because the marginal inference cost on a single-class

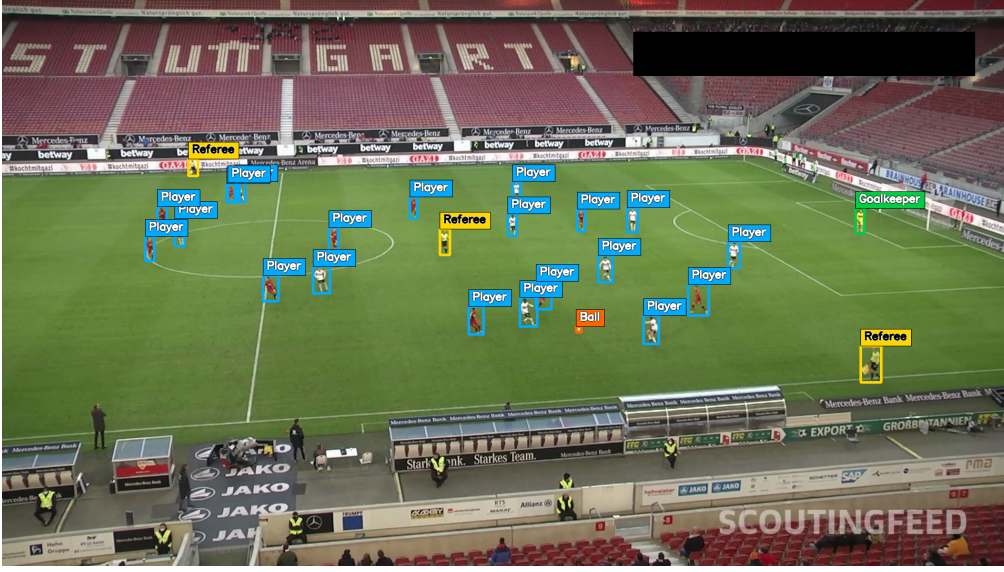


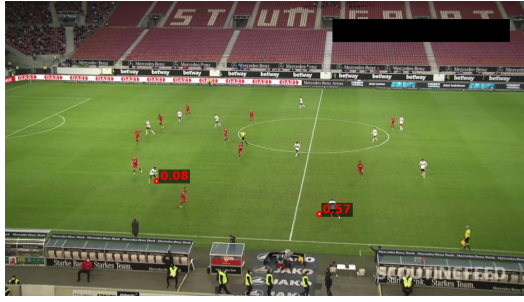
Figure 3.2: Representative output of the generic four-class detector. The model localises players, goalkeepers and referees in broadcast footage, providing the person detections used by the downstream tracking and team-classification stages. Although the detector also predicts a ball class, ball localisation in the final pipeline is handled by the dedicated ball detector.

head is small, and larger backbones help on fine-grained small-object features. Training used the same 1280×1280 resolution and augmentation regime as the generic detector, ran for 150 epochs at batch size 8, and completed in approximately 110 minutes. The resulting model achieves test mAP₅₀ of 0.950 and recall 0.896 — a 49 percentage-point absolute improvement over the generic detector’s ball performance (Table 3.3).

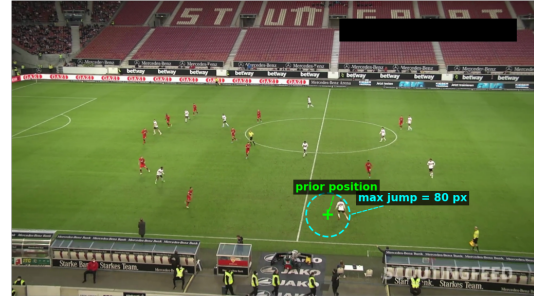
At inference, the detector runs at a deliberately permissive confidence threshold ($\text{conf}=0.05$) so legitimate low-confidence detections survive into temporal filtering. From these candidates, the filter selects the detection nearest the previous frame’s ball centre, subject to a maximum displacement of 80 pixels and a minimum confidence of 0.25. The 80-pixel gate constrains per-frame ball motion at broadcast frame rates and resolutions; the 0.25 minimum is the deferred confidence gate that the permissive detector threshold leaves to post-processing. This filter relies on a high-recall candidate set the generic detector cannot reliably provide.

Figure 3.3 illustrates this candidate-gating process.

The generic detector runs at confidence 0.2 with NMS IoU threshold 0.5, following Equation 2.3 and the procedure of Section 2.4.1. The 0.2 threshold is deliberately low: the tracker performs additional quality gating during association, and recovering a



(a) Frame t — all candidates ($\text{conf} \geq 0.05$).



(b) Frame $t - 1$ — prior position and displacement gate.



(c) Frame t — selected position after temporal filtering.

Figure 3.3: Temporal coherence filtering for the dedicated ball detector. A permissive detector threshold preserves multiple candidate ball detections, after which the previously accepted ball position defines an 80-pixel displacement gate. The candidate that remains both spatially plausible and sufficiently confident is retained as the ball observation for frame t . This post-processing step improves recall while suppressing isolated false positives before possession, pass and offside reasoning are applied.

marginal detection it can stabilise across frames is preferable to losing it at the detector stage.

Detector	P	R	F1	mAP ₅₀	mAP _{50–95}
Generic (Ball class)	0.818	0.393	0.531	0.459	0.194
Dedicated ball	0.964	0.896	0.929	0.950	0.556

Table 3.3: Ball-class performance on the held-out test set: generic four-class detector versus dedicated single-class detector. The dedicated detector raises ball mAP₅₀ from 0.459 to 0.950 — a 49 percentage-point absolute improvement that motivates the two-detector architecture. Per-class metrics for all classes of the generic detector are reported in Chapter 4.

3.5 Multi-Object Tracking

Detector outputs are stateless, but every downstream tactical output depends on stable identities across frames. The pipeline meets this need with a multi-object tracker, addressing FR2 and Objective 2 (Section 1.2).

The tracker follows the tracking-by-detection paradigm of Section 2.5.1: per-frame detections are associated with active tracks by solving a bipartite assignment problem on a cost matrix derived from spatial overlap between predicted and observed boxes. The pipeline invokes this through Ultralytics’s unified `model.track()` interface, which wraps the detector call and applies the configured association algorithm in-place. The default tracker is ByteTrack [49], with BoT-SORT [2] available as a drop-in alternative through a configuration flag (`--tracker bytetrack.yaml` or `botsort.yaml`). The two algorithms differ in their association strategies (Sections 2.5.3 and 2.5.4); supporting both lets Chapter 4 compare them like-for-like against SC (ii), $\text{HOTA} \geq 0.45$.

Both trackers run with their Ultralytics-shipped configurations unmodified, in line with Section 3.3: tracker-internal hyperparameters are well-studied [49, 2] and tuning them was deliberately out of scope. Holding the published defaults also makes the comparative evaluation a fair baseline rather than a hyperparameter sweep.

Per frame, the tracker emits identified detections each carrying a class label, bounding box, confidence score, and stable track identifier. The per-frame loop splits these by class: player and goalkeeper tracks feed the team classifier of Section 3.6, while referee tracks bypass it because referees belong to neither team and are excluded from possession reasoning. The ball is handled outside the tracker because at most one ball is ever on the pitch; the temporal coherence filter of Section 3.4 provides the equivalent single-target stabilisation.

Figure 3.4 shows a qualitative example of track identity continuity before, during and after close player interaction.

3.6 Team Classification

Producing per-player team labels without manual annotation addresses Objective 3 (Section 1.2) and FR3. The problem is unsupervised: each match brings a novel pair of kits, no labelled training data is available, and the only signal is visual appearance. The pipeline uses the SigLIP-UMAP- k -Means formulation of Section 2.6

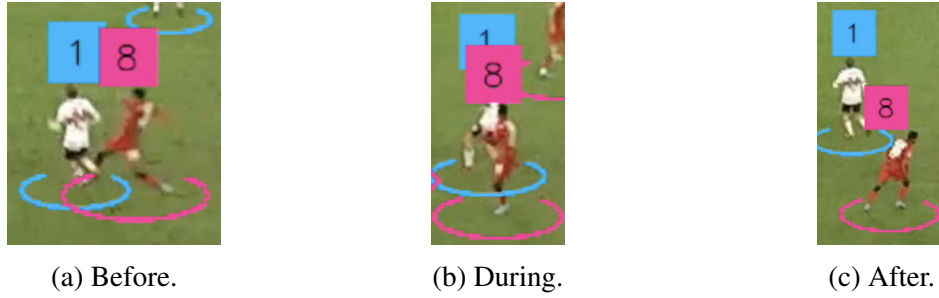


Figure 3.4: Tracking continuity through close player interaction. The same two track identities are retained before, during and after a short overlap between players, illustrating the identity stability required by downstream modules such as team-label smoothing, possession tracking and heatmap accumulation.

via the open-source `sports` library [41], which supplies reference implementations of embedding extraction, dimensionality reduction, and clustering. The contribution here is the surrounding architecture that makes these building blocks usable inside a per-frame pipeline: a one-shot warmup phase, torso-focused cropping, track-level temporal smoothing, and a separate goalkeeper resolver.

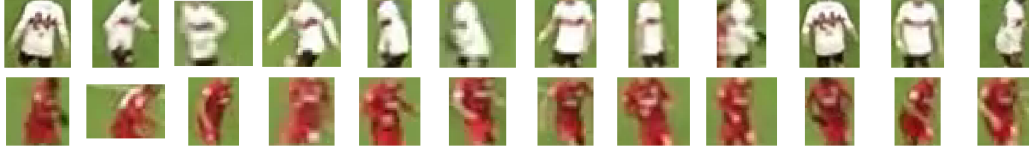
3.6.1 Warmup-Phase Integration

The clusterer of Equation 2.15 is fitted once per video, in a warmup phase preceding the per-frame loop. The warmup samples frames at a fixed stride (`warmup_stride=30`, roughly one frame per second at 30 fps) over the first `warmup_seconds=20` of footage. For each sampled frame the generic detector of Section 3.4 is run, only player-class detections are retained, boxes smaller than 35×35 pixels are discarded as unreliably small, and the survivors are torso-cropped (Section 3.6.2). Crops are capped at `max_warmup_crops=800` and passed in a single batch to `TeamClassifier.fit()`, which extracts SigLIP embeddings, projects them through UMAP, and fits a $k = 2$ k -Means model.

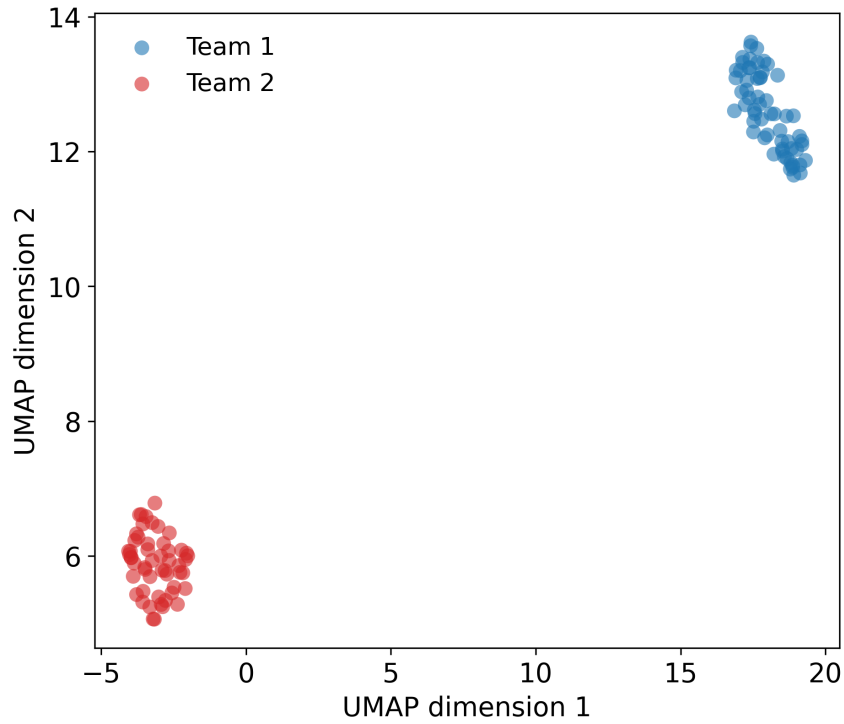
Fitting once rather than incrementally is methodologically central. k -Means is sensitive to the empirical distribution seen during fitting, so incremental or per-frame re-fitting would produce a moving cluster boundary — a player whose embedding sat fractionally on one side in frame t could be reassigned in frame $t + 1$ purely because the boundary moved, not because appearance changed. Fitting once on a representative twenty-second window stabilises the boundary for the rest of the video. The trade-off is no adaptation to sudden visual change, such as a substitute in a markedly different kit, but in broadcast football this is benign: kits do not change mid-match,

and twenty seconds suffices for both teams to appear under typical broadcast direction. The 800-crop cap is an empirical compromise between fit stability and the embedding-extraction latency that dominates warmup time.

Figure 3.5 shows representative torso crops and the resulting UMAP- k -Means separation.



(a) Representative warmup torso crops.



(b) UMAP projection of SigLIP embeddings with $k = 2$ k -Means labels.

Figure 3.5: Warmup-phase diagnostic for team assignment. (a) Representative torso crops sampled during the warmup period. (b) The corresponding SigLIP embeddings after UMAP projection, coloured by the resulting $k = 2$ k -Means labels. The clear separation between the two groups supports the use of a per-video warmup phase to initialise team assignment before track-level temporal smoothing and goalkeeper resolution are applied.

3.6.2 Torso Cropping

Embedding-model crops are the upper-body subregion of each player’s bounding box, taken between vertical fractions `torso_top=0.15` and `torso_bottom=0.65`. The motivation is signal-to-noise: jersey colour, sponsor logo, and crest — the discriminative cues for team identity — appear in the torso, while the lower body contributes shorts (often shared between home and away kits) and grass that varies with camera angle rather than team membership. Excluding the top fraction removes the head and a thin halo of background; excluding the bottom fraction removes this lower-body noise. The restricted band gives the SigLIP encoder a more homogeneous input distribution and reduces the risk of the model learning spurious lower-body or background features as cluster cues.

3.6.3 Track-Level Temporal Smoothing

Per-frame predictions are noisy. Even with a well-conditioned cluster fit and clean crops, individual frames can produce wrong assignments due to motion blur, partial occlusion, an unusual pose, or a player turning to hide the jersey. Yet team identity is a constant property of a track — a player does not change teams during a match. The `TrackMajorityVote` class in `assigner.py` exploits this by maintaining, for each track identifier, a deque of the last `team_smooth=30` predictions (approximately one second at 30 fps) and emitting the modal value as the smoothed label.

This window is short enough to react quickly when a track identifier is replaced (when the underlying tracker drops a track and a new identifier takes over), but long enough that transient errors are out-voted by surrounding correct predictions. Smoothing is applied at the track level rather than per detection, so detections arriving without a track identifier fall back to the raw per-frame prediction.

3.6.4 Goalkeeper Resolution

k -Means is fitted with $k = 2$ for the two outfield teams. Competition rules require goalkeeper kits to be visually distinct from both outfield kits and from each other, so neither fitted cluster reliably corresponds to a goalkeeper, and a goalkeeper’s embedding can fall on either side of the boundary by chance. A $k = 3$ fit was rejected: each match contains only two goalkeepers, the warmup window typically captures few goalkeeper crops, and $k = 3$ on a heavily two-team-dominated crop set is poorly conditioned.

Goalkeeper identity is instead resolved by a spatial heuristic in `gk_resolver.py`. For each frame, the bottom-centre image-space coordinate of every outfield team-0 player is averaged to give a team-0 centroid, and likewise for team 1. Each goalkeeper is assigned to the nearer centroid. The reasoning is positional: goalkeepers spend most of the match near or behind their own team’s most defensive outfield players. Crucially the heuristic operates in image-space, not pitch-space, so it continues to work during frames where the homography of Section 3.7 cannot be updated for lack of confident keypoints. The implicit assumption — that the two teams’ centroids project to distinguishable image-space locations — holds for the side-on tactical camera angles that dominate broadcasts.

Figure 3.6 illustrates this image-space assignment.

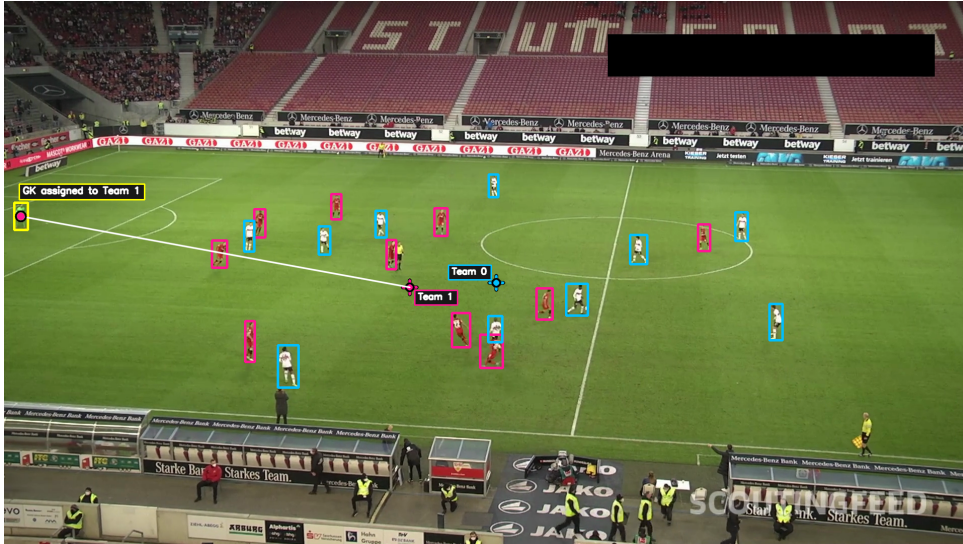


Figure 3.6: Image-space goalkeeper resolution on a broadcast frame. Outfield players are first assigned to teams and each team’s image-space centroid is computed. The goalkeeper is then assigned to the nearest outfield-team centroid, avoiding reliance on goalkeeper kit colour for team classification.

Whether this team-classification stack meets SC (iii) ($\text{macro-}F_1 \geq 0.85$) is reported in Chapter 4.

3.7 Pitch Registration

Pitch registration is the geometric stage that maps image-space detections into pitch coordinates, providing the substrate for all tactical analytics. The pipeline addresses this in two parts: a YOLO-pose model trained to localise a fixed set of pitch landmarks,

and a homography estimator that consumes them via the Direct Linear Transform of Section 2.7.2. Together they address FR4 and Objective 4 (Section 1.2).

3.7.1 Keypoint Detector

A YOLOv8x-pose model [17] is fine-tuned to detect a single class (*pitch*) annotated with 32 keypoints at geometrically distinctive features of the pitch markings: line intersections, penalty-area and six-yard-box corners, halfway-line endpoints, and tangencies of the centre circle and penalty arcs. The landmark set follows the broadcast- football registration literature [46, 45]; each keypoint has a known pitch-space position, so any subset detected with sufficient confidence yields image-to-pitch correspondences directly usable in DLT.

Figure 3.7 shows the 32-point pitch template used for these correspondences.

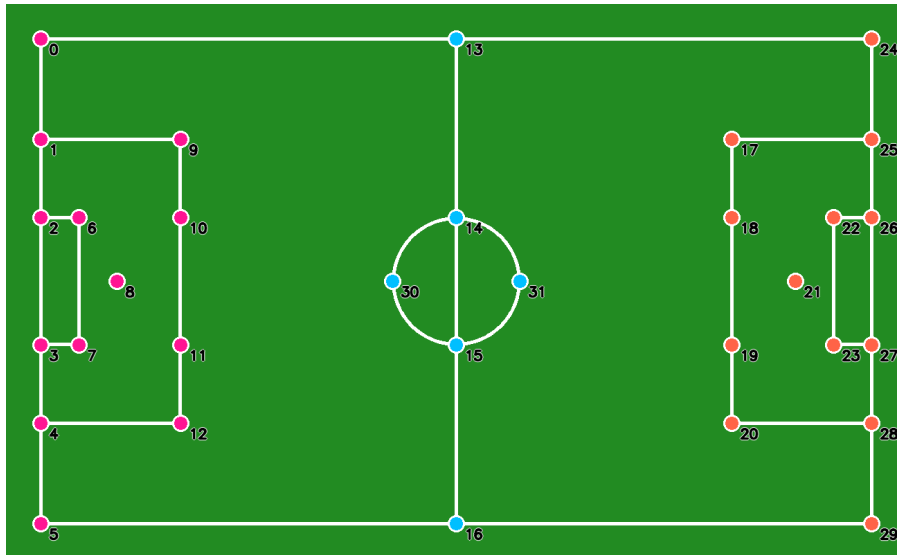


Figure 3.7: Pitch-keypoint template used for registration. The 32 landmarks correspond to stable pitch-line intersections, penalty-area corners, halfway-line points, and centre-circle points with known pitch-space coordinates. The fixed correspondence between detected image-space keypoints and this pitch-space template supplies the point pairs used by the DLT homography estimator.

The detector was trained on the *football_field_detection_dataset* (Roboflow), comprising 317 broadcast frames split 255/34/28 across train/validation/test, on the same A100 hardware and Ultralytics 8.4.21 stack used for the detection models of Section 3.4. Full hyperparameters are catalogued in Section 3.10.3; one non-default

choice is methodologically substantive enough to surface here. Mosaic augmentation (`mosaic=0.0`, disabled) concatenates four training images into a single synthetic input — well-motivated for general object detection, where it exposes the model to varied scales and contexts, but it fragments the global geometric structure of a pitch into four disjoint quadrants. The discriminative signal for pitch-keypoint localisation is precisely the spatial relationships between landmarks — a corner flag is identifiable in part by its position relative to the goal line and touchline — so fragmenting that structure would train the model on geometries that never appear at inference. Disabling mosaic was therefore a deliberate departure motivated by the structural mismatch between the augmentation and the fixed-landmark task.

The trained model achieves test detection mAP_{50} of 0.995 (the pitch is, in practice, always present in broadcast frames) and test pose mAP_{50} of 0.995 with mAP_{50-95} of 0.953. Per-keypoint mean errors cluster around four pixels, approximately 0.4% of the image diagonal. One keypoint (`kp_16`) is an outlier at 18.2 pixels, roughly four times the median; this is reported here for transparency and its effect on homography stability is discussed in Chapter 4.

Figure 3.8 illustrates the registration step end-to-end.

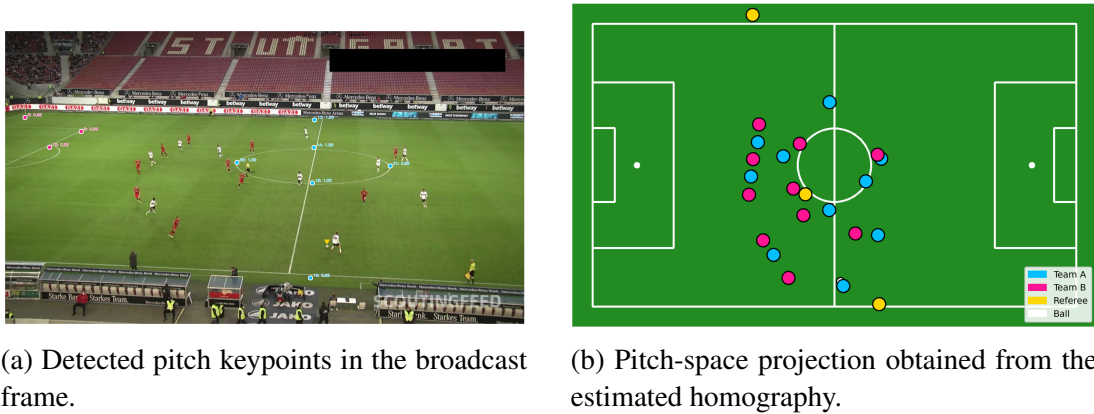


Figure 3.8: Pitch registration in action. Detected pitch keypoints are used to estimate a homography via Direct Linear Transform, enabling image-space detections to be projected into pitch coordinates for bird’s-eye-view rendering and downstream tactical analysis.

3.7.2 Stride-Cached Homography Estimation

Pitch keypoint inference is the slowest pipeline stage by a substantial margin: a separate forward pass through a large pose model served via a remote endpoint. Running it

every frame would dominate end-to-end latency, but two observations make per-frame inference unnecessary. First, broadcast camera motion is slow relative to frame rate — a tactical camera pans over hundreds of milliseconds, not single-frame intervals. Second, a homography estimated from a stable keypoint set is itself robust to small camera movements. The pipeline exploits this by running keypoint inference every `pitch_stride=15` frames, caching the resulting `ViewTransformer`, and reusing it on the intervening frames. At 30 fps the homography is recomputed twice per second, well above the rate of meaningful camera motion in broadcast football.

When inference runs, three thresholds gate the construction of a new homography. Each predicted keypoint must clear a per-keypoint confidence of `kp_conf=0.5`; the resulting confident set must contain at least `min_good_kps=6` entries; and inference runs with a model-level confidence floor of 0.3. The 6-keypoint minimum is the most consequential. The DLT system of Equation 2.18 requires four non-collinear correspondences for a unique homography; six provides redundancy that the least-squares solve uses to suppress per-keypoint localisation noise without becoming numerically fragile. If insufficient confident keypoints are present, the previous homography is retained — the camera has not moved significantly in the intervening half-second, so the cached transform remains a reasonable approximation.

The `ViewTransformer` exposes a vectorised `transform_points` method mapping any image-space coordinate into pitch-space via the projective transformation of Equation 2.16 followed by the dehomogenisation of Equation 2.17. Player anchors and the ball are projected through this method every frame, supplying the pitch-space coordinates that Section 3.8 consumes. Whether the registration meets SC (iv) (median pitch-space reprojection error ≤ 250 cm) is reported in Chapter 4.

3.8 Tactical Reasoning

The four preceding sections supply, on every frame, tracked players with team labels and confident pitch-space correspondences. Tactical reasoning turns these intermediates into the analytical outputs that motivate the project: bird’s-eye trajectories, heatmaps, speed and distance, possession, pass events, and offside flags. All modules operate in pitch-space coordinates, in line with the ground-plane validity argument of Section 2.8.1, and are presented in the order in which they consume each other’s outputs. The section addresses FR5 and Objective 5 (Section 1.2), and supplies the pass and offside event logs against which SC (v) and SC (vi) are evaluated in Chapter 4.

3.8.1 Pitch-Space Projection

For each tracked player, the bottom-centre of the bounding box is the ground-plane anchor and is projected through the cached homography using the `ViewTransformer` of Section 3.7, following Equation 2.21. The bottom-centre choice is the standard broadcast-analytics convention: it approximates the point where the player’s feet meet the playing surface, the unique location on the bounding box that lies on the ground plane and therefore satisfies the homography’s planar assumption. The ball is projected from its detected centre. All downstream modules consume these pitch-space coordinates, expressed in centimetres with the pitch origin at one corner.

3.8.2 Possession Tracking

Possession is assigned by nearest-eligible-player matching, in the formulation of Equation 2.27: for each frame with a ball detection, the player whose pitch-space anchor lies closest to the ball is identified, and possession is assigned to that player provided the distance is at most `proximity_cm=350` cm. Referees are excluded from the eligibility set using the class labels defined in Section 3.6, since they should never be considered in possession. If no eligible player lies within the threshold, possession for that frame is left unassigned rather than attributed to the nearest-but-still-distant player. The 350 cm threshold absorbs the combined effect of bounding-box anchor error and homography projection noise, which together can shift a player’s estimated pitch position by tens of centimetres even with a correct underlying detection, while remaining small enough to disambiguate possession in the typical congested-midfield scene.

3.8.3 Pass Detection

Pass detection sits on top of possession assignment. Per-frame possession labels are inherently noisy: a frame with the ball in flight between two players can briefly attribute possession to a third who happens to lie near the trajectory, and a failed ball detection can break a stable possession run. The pipeline addresses this with a hold-stability requirement: a player is considered to genuinely possess the ball only after the per-frame label has named that player for at least `min_hold_frames=3` consecutive frames, and a pass is registered only when stable possession transitions from one player to a distinct other. Transitions that fail the stability requirement on either side are discarded as transient noise.

The detection logic is implemented in the `_PossessionTracker` class in `offside.py`. A transition between two players on the same team is logged as a *pass*; a transition between players on opposing teams as an *interception*. Both are written to the JSON event log with the schema:

```
{ "frame_idx": int,
  "passer_track_id": int,
  "passer_team_id": int,
  "ball_pitch_xy": [float, float] }
```

3.8.4 Offside Detection

Offside detection extends the pass-detection module with the geometric criterion of Equation 2.28. The pipeline implements *positional* offside only — the spatial component of Law 11 — and does not attempt the active-involvement clause, which depends on behavioural cues that broadcast video alone does not reliably expose.

Two pieces of context are required: the attacking direction of each team and the location of the second-last defender at the moment a pass is initiated. Attacking direction is recovered automatically by `_AttackDirectionEstimator`, which accumulates pitch-space x -coordinates of each team over the first `direction_min_samples=30` sampled frames and assigns the team with the lower mean x as attacking right (and vice versa). The estimator runs once early in the video and the result is fixed thereafter; mid-half direction switches are out of scope and are recorded as a known limitation in Section 3.11. When the pass-detection module fires, the offside checker locates the defending team's second-last defender along the attacking x -axis and flags the receiver as offside if their pitch-space x -coordinate is beyond the defender's by more than `offside_margin_cm=30` cm. The margin absorbs combined detection-and-projection noise, preventing flags on plays where a player appears fractionally beyond the defender purely because of pixel-level anchor error compounded by homography uncertainty.

Figure 3.9 shows the offside decision in both the annotated broadcast frame and the corresponding pitch-space representation.

Offside events trigger a visual annotation persisting for `display_frames=50` frames (approximately 1.7 s at 30 fps): the offending player's box is drawn red on the broadcast frame, marked red on the bird's-eye view, and a dashed red line is drawn at the offside x -coordinate to indicate the defender's line. Each event is logged with the

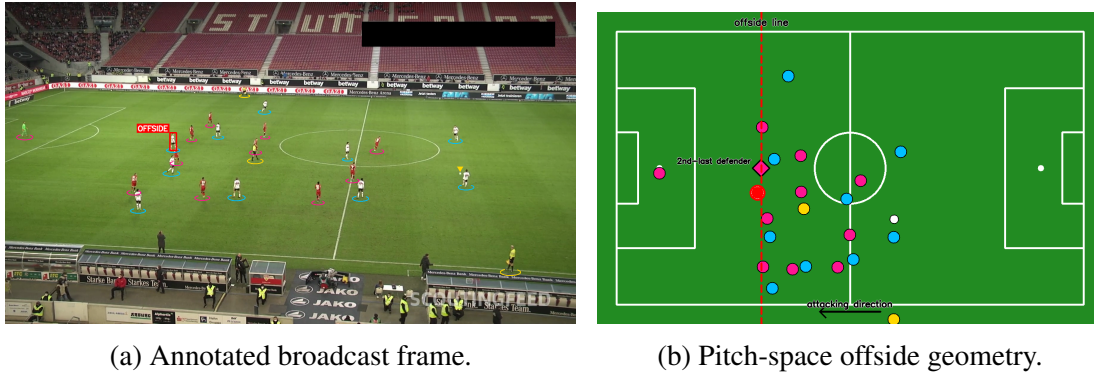


Figure 3.9: Offside decision visualised in image space and pitch space. The broadcast frame highlights the player flagged as offside, while the bird’s-eye view shows the attacking direction, the offside line, and the second-last defender, marked by a diamond symbol, used by the positional offside checker.

schema:

```
{ "frame_idx": int,
  "pass_event": { ... pass-event fields above ... },
  "offside_line_x": float,
  "offside_players": [
    { "track_id": int, "pitch_xy": [float, float] }, ...
  ] }
```

3.8.5 Speed and Distance

Per-track speed and cumulative distance are computed by finite-differencing the pitch-space anchor across consecutive frames, following Equations 2.24 and 2.26. Speed is reported in km/h and distance in metres in the per-player stats JSON (`distance_m`, `avg_speed_kmh`, `max_speed_kmh`). A plausibility cap (`max_speed_kmh=40`) rejects per-frame displacements implying speeds beyond this threshold: elite sprinters peak around 35–37 km/h, so 40 km/h leaves headroom above the realistic ceiling while still catching single-frame jumps from tracking and projection noise. Frames whose finite-differenced speed exceeds the cap have their displacement zeroed, preserving cumulative distance against noise spikes at the cost of also rejecting genuine motion above 40 km/h — a regime that does not occur in football.

3.8.6 Heatmaps

Per-track positional heatmaps are accumulated on a pitch-space grid, following the histogram construction of Equation 2.22. The grid defaults to 480×280 bins covering the full pitch, giving a spatial resolution of roughly 20×25 cm per bin — finer than the projection-pipeline noise floor and coarser than per-frame jitter, so each bin aggregates many frames of presence rather than amplifying noise. On each frame the bin containing each player’s pitch-space anchor is incremented. At the end of the video, each track’s histogram is post-processed: Gaussian smoothing as in Equation 2.23, followed by $\gamma = 0.22$ correction to compress dynamic range and reveal less-visited regions. The γ correction is a display choice rather than a statistical operation: the raw histogram is also written to disk as an NPY array so downstream analysis can operate on the unsmoothed counts.

Figure 3.10 shows projected track samples being converted into the displayed heatmap.

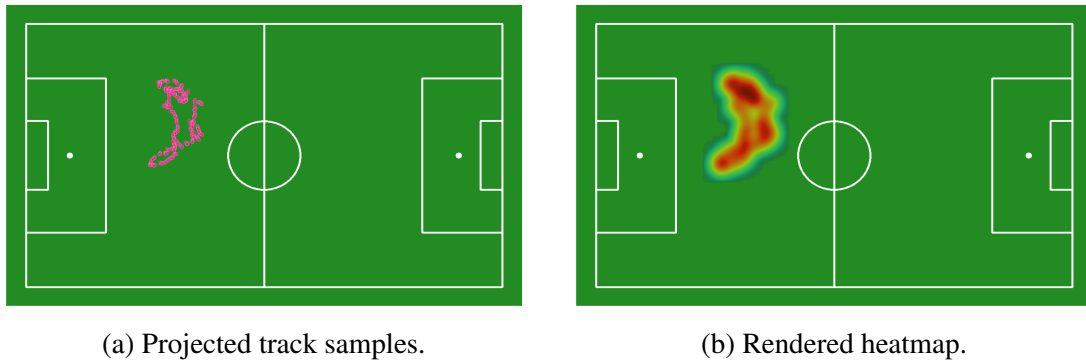


Figure 3.10: Per-track heatmap construction. Projected pitch-space positions are accumulated in a track-specific occupancy grid and rendered with smoothing and γ correction for visual readability.

3.9 Implementation Testing and Verification

Section 3.3 emphasised component-level isolation because errors in any upstream stage propagate through every downstream tactical output, making per-component verification on representative input essential rather than optional.

The verification performed during development is summarised in Table 3.4. These procedures do not constitute formal automated testing in the sense of a continuous-integration suite; they are the deliberate inspection steps the development loop produced as each component was integrated. Quantitative evaluation of the integrated

system against the success criteria of Section 1.2 is reported separately in Chapter 4.

3.10 System Engineering and Reproducibility

3.10.1 Pipeline Orchestration

The modules of the preceding sections are integrated through a single orchestration script (`run.py`) that exposes the pipeline as a command-line tool. The implementation follows a modular layout under `src/`, with one subpackage per processing stage — `ball/`, `heatmap/`, `pitch/`, `stats/`, `team/`, and `pipeline/` — so each component can be modified, tested, or replaced independently. The tracker swap is the cleanest instance: switching between ByteTrack and BoT-SORT requires only a different configuration filename passed to the Ultralytics tracker interface, with no changes elsewhere in the codebase, satisfying NFR1.

Execution proceeds in two phases. The warmup phase of Section 3.6.1 runs once at the start of each video to fit the team-classification model on a representative crop set; the per-frame loop then iterates through the remaining frames in the order of Figure 3.1. The two-phase structure is required by the cluster-stability argument of Section 3.6.1 and provides a natural separation between fitting diagnostics and per-frame output.

Several caching layers amortise computational cost: pitch keypoint inference is cached every fifteen frames (Section 3.7.2); the previous ball position is held as a one-frame cache for the temporal coherence filter (Section 3.4); per-track team-prediction history is held in a thirty-frame deque (Section 3.6.3); and per-track heatmap arrays accumulate in memory until end-of-video. The heatmap arrays are the largest cache at 480×280 floats per active track but remain modest in absolute terms. Therefore, the pipeline runs in a single Python process without explicit memory management.

3.10.2 Output Formats

The output format reflects the dual audience for the pipeline’s results, satisfying NFR3. An annotated MP4 and bird’s-eye-view MP4 are produced for human inspection alongside per-track heatmap PNGs. For machine consumption, raw heatmap counts are exported as NPY arrays, and detected pass and offside events together with per-player match statistics are serialised to JSON using the schemas of Section 3.8. These JSON outputs are the inputs Chapter 4 consumes for the quantitative pass-detection assessment of SC (v); the human-inspectable visualisations support the qualitative coherence

Component	Verification performed	Purpose
Detection pipeline	Visual inspection of bounding-box overlays on sampled frames; class-filter and confidence-threshold behaviour confirmed against schema.	Confirms downstream modules receive valid, correctly-classed detections.
Ball filter	Spot checks on clips with detection gaps and on clips with rapid ball motion; verified displacement-gate rejection produced expected behaviour.	Confirms the temporal filter rejects implausible candidates without suppressing legitimate motion.
Team classifier (warmup)	Inspection of the fitted UMAP projection and k -Means cluster boundary on the warmup crop set; confirmed both teams recovered as separated clusters.	Confirms the warmup fit produces a usable two-team boundary before per-frame prediction begins.
Track-level smoothing	Comparison of raw per-frame versus smoothed team labels for a sample of tracks across development clips.	Confirms smoothing reduces frame-level noise without lagging genuine track-identifier changes.
Homography estimator	Verified that projected pitch coordinates of detected players fall within pitch bounds on the majority of frames; manual sanity-check of the bird's-eye view.	Detects ill-conditioned or failed homography fits that would otherwise corrupt downstream analytics.
Possession module	Frame-level inspection of nearest-player assignments on possession-of-ball sequences from development clips.	Confirms proximity-based assignment behaves as intended in congested midfield scenes.
Pass and offside detection	Manual inspection of detected pass and offside events against the source footage.	Distinguishes genuine passes from possession noise; confirms the offside criterion fires only at pass moments.
Output generation	Verified that all expected output categories (annotated MP4, bird's-eye MP4, heatmap PNGs, JSON event log and stats) are produced for each run-mode preset.	Ensures end-to-end pipeline completeness across all modes.

Table 3.4: Component-level verification performed during development. Each row describes a deliberate inspection step performed before the corresponding component was relied on in the integrated pipeline. Quantitative evaluation is reported separately in Chapter 4.

assessment of SC (vi).

Operational ergonomics are handled by a wrapper shell script (`run.sh`) that exposes mode presets — `preview` for short test clips, `full` for end-to-end processing of complete matches, and `debug` for verbose intermediate logging — so common invocations do not require remembering the underlying argument set. A separate Streamlit interface provides a graphical front-end accepting a video upload and surfacing the same outputs as the command-line entry point, demonstrating that the pipeline can be packaged for use beyond the development environment.

3.10.3 Reproducibility

Three categories of reproducibility commitment apply, satisfying NFR2.

Environment. Training was carried out on an NVIDIA A100-SXM4-80GB GPU using Python 3.11, Ultralytics 8.4.21, PyTorch 2.8.0, and CUDA 12.8; inference is developed against an Apple M2 Pro (no discrete GPU). The pipeline runs end-to-end in a single Python process without multi-GPU coordination.

Models, datasets, and configuration. Three YOLO models are fine-tuned: the four-class generic detector and the dedicated ball detector (Section 3.4), and the pitch-keypoint detector (Section 3.7). Each is trained on a public Roboflow dataset (*Football_object_detection_dataset*, *ball_dataset*, and *football_field_detection_dataset* respectively). Trained weights are saved locally; the keypoint detector is additionally served via Roboflow’s serverless inference endpoint at runtime, with local deployment of the trained weights supported as a fallback. All pipeline-level hyperparameters discussed in this chapter are exposed as command-line arguments by `run.py`, with mode-specific defaults in `run.sh`. The training hyperparameters of the three fine-tuned YOLO models are recorded in Table 3.5; all values follow the Ultralytics 8.4.21 defaults except where they differ between models or where the methodology rationale required a departure (the disabling of mosaic augmentation for the keypoint detector, argued in Section 3.7).

Limitations. Training was seeded (`seed=0`) and `deterministic=true` was set, so re-running training on the same hardware and software stack should yield numerically identical weights. Bit-exact reproduction across different hardware or CUDA versions is nonetheless not guaranteed because some CUDA kernels remain non-deterministic

Hyperparameter	Generic	Ball	Keypoint
<i>Model and data</i>			
Architecture	YOLOv8m	YOLOv8x	YOLOv8x-pose
Pretrained backbone	COCO	COCO	COCO
Input resolution	1280 ²	1280 ²	640 ²
<i>Optimisation</i>			
Optimiser	auto	auto	auto
Initial LR (<code>lr0</code>)	0.01	0.01	0.01
Final LR factor (<code>lrf</code>)	0.01	0.01	0.01
Momentum	0.937	0.937	0.937
Weight decay	5×10^{-4}	5×10^{-4}	5×10^{-4}
Warm-up epochs	3	3	3
Epochs	50	150	500
Early-stop patience	100	30	100
Batch size	4	8	48
Mixed precision (AMP)	enabled	enabled	enabled
<i>Loss weights</i>			
Box / Class / DFL gains	7.5 / 0.5 / 1.5	7.5 / 0.5 / 1.5	7.5 / 0.5 / 1.5
Pose / KObj gains	—	—	12.0 / 1.0
<i>Augmentation</i>			
Mosaic	1.0	1.0	0.0 (disabled)
Close-mosaic epochs	10	10	10
HSV (<i>h</i> / <i>s</i> / <i>v</i>)	0.015 / 0.7 / 0.4	0.015 / 0.7 / 0.4	0.015 / 0.7 / 0.4
Translate / Scale	0.1 / 0.5	0.1 / 0.5	0.1 / 0.5
Horizontal flip	0.5	0.5	0.5
RandAugment / Erasing	on / 0.4	on / 0.4	on / 0.4
<i>Reproducibility controls</i>			
Random seed	0	0	0
Deterministic mode	enabled	enabled	enabled

Table 3.5: Training hyperparameters for the three fine-tuned models. The `auto` optimiser setting lets Ultralytics select between SGD and AdamW based on the total number of training iterations. Per-model differences are deliberate: the ball detector uses a heavier backbone and longer training to compensate for the small-object task, while the keypoint detector trains at a lower input resolution with a larger batch and disables mosaic augmentation, as argued in Section 3.7.

at the device level; in practice this produces numerically distinct but behaviourally equivalent weights. Independently, the Roboflow-hosted inference endpoint is a third-party service whose continued availability cannot be guaranteed; local deployment of the trained keypoint weights is supported as a fallback for replicators without access to the hosted endpoint.

3.11 Design Decisions and Failure Handling

The preceding sections justified each design decision in the context where it was made. This section consolidates them, alongside the system’s known failure modes, to make the overall design rationale and its limitations visible at a glance. Table 3.6 lists the principal architectural decisions with their considered alternatives and trade-offs; Table 3.7 records the failure conditions the pipeline is designed to absorb gracefully, the mitigation in each case, and the residual limitation the mitigation does not fully resolve.

The decisions and limitations above scope the quantitative evaluation of Chapter 4: the pipeline is designed to absorb these failure modes gracefully rather than eliminate them, and the success criteria of Section 1.2 are assessed against this scoped behaviour.

Decision	Alternative	Rationale	Trade-off
Two-detector architecture	Single multi-class detector	49 pp ball-class mAP ₅₀ improvement on the same task (§3.4).	Second inference pass per frame.
Mosaic disabled for keypoint training	Default Ultralytics augmentation	Pitch geometry has fixed structure that mosaic fragments into disjoint quadrants (§3.7).	Fewer augmented training samples; longer training to convergence.
Stride-15 cached homography	Per-frame keypoint inference	Keypoint inference dominates pipeline latency; broadcast camera motion is slow (§3.7).	Brief projection drift during rapid camera moves.
$k = 2$ team clusterer + spatial GK resolver	Three-cluster team model	$k = 3$ on a heavily two-team-dominated crop set is poorly conditioned (§3.6).	Requires a separate goalkeeper-handling pathway.
Fit-once warmup clustering	Per-frame or incremental refit	Cluster boundary stability; avoids label flip-flop from drifting boundary (§3.6.1).	No adaptation to mid-match visual change.
Track-level team smoothing	Per-detection prediction	Team identity is a constant property of a track; absorbs frame-level prediction noise (§3.6.3).	One-second latency in reacting to genuine track-identifier changes.
Positional offside only	Full Law 11 modelling including active involvement	Active involvement depends on behavioural cues that broadcast video does not reliably expose (§3.8.4).	Cannot distinguish active from passive offside positions.
Image-space GK centroid resolution	Pitch-space resolution	Functions before homography is valid; preserves goalkeeper assignment when keypoints are insufficient (§3.6.4).	Implicit assumption of side-on broadcast camera angle.

Table 3.6: Principal architectural decisions, their considered alternatives, and the trade-offs each decision accepts.

Failure condition	Mitigation strategy	Residual limitation
Ball not detected in a frame	Possession is left unassigned for that frame rather than imputed.	Passes may be missed during sustained ball-detection gaps.
Implausible ball position jump	Temporal-coherence filter rejects the candidate; previous ball position is retained.	Genuine high-speed ball motion can be suppressed alongside noise.
Insufficient confident pitch keypoints	Previous cached homography is retained rather than replaced.	Projection drift accumulates during sequences of failed inference.
Track identifier lost or replaced	Track-level smoothing absorbs short label noise; new identifier starts fresh.	Per-track heatmaps and statistics may fragment when tracks split.
Goalkeeper visually similar to an outfield kit	Spatial GK resolver assigns to nearest team centroid in image-space.	Fails for behind-the-goal camera angles.
Half-time attacking-direction switch	Direction estimator runs once early; result is fixed thereafter.	Manual intervention or a re-run is required to handle full-match clips.
Single-frame anomalous speed (e.g. track-id swap)	40 km/h plausibility cap zeroes the displacement contribution for that frame.	Genuine motion above the cap is also rejected, though this regime does not occur in football.

Table 3.7: Failure conditions handled by the pipeline, the corresponding mitigation strategy, and the residual limitation that the mitigation does not fully address. The half-time direction-switch row resolves the forward reference made in Section 3.8.4.

Chapter 4

Evaluation

4.1 Evaluation Methodology

This chapter evaluates the pipeline against the six success criteria committed in Chapter 1: detector $\text{mAP}_{50} \geq 0.80$, tracking $\text{HOTA} \geq 0.45$, team-assignment $\text{macro-}F_1 \geq 0.85$, median pitch-space reprojection error ≤ 250 cm, pass-detection $F_1 \geq 0.80$, and qualitatively coherent bird’s-eye, heatmap, and offside outputs. Each criterion is addressed in the section covering the relevant pipeline stage, with explicit verdicts where the evidence permits.

Evaluation operates at three levels of evidence. *Component-level* evaluation (Sections 4.2 and 4.5) uses the held-out test splits of the training datasets and reports the standard object-detection and pose metrics defined in Sections 2.4.1 and 2.5.5, allowing direct comparison with the broadcast-football literature [5, 46, 31]. *Integration-level* evaluation (Sections 4.3 and 4.4) characterises stages whose behaviour depends on temporal context — tracking continuity and team-label stability — using a combined SportsMOT-football [6] and broadcast tracking set, and three independent clips with manually labelled torso crops for team assignment. *System-level* evaluation (Sections 4.6 and 4.7) examines end-to-end outputs — event logs, per-player statistics, heatmaps, runtime — on full pipeline runs against the same three clips, with event ground truth obtained by visual inspection of the annotated outputs.

The evaluation is scoped by the failure-handling table of Section 3.11: claims are made only within its operational regime, and degraded-input behaviour is characterised in Section 4.8 rather than treated as out-of-scope. Omitted metrics or comparisons are flagged and justified explicitly.

4.2 Detection Performance

The two detectors are evaluated on held-out test splits (75 images / 1,875 instances for the generic detector; 119 images / 119 instances for the ball detector) using the COCO-style protocol of Section 2.4.1. Weights were selected by validation mAP_{50-95} (epoch 40 generic, epoch 143 ball).

4.2.1 Generic four-class detector

The generic detector achieves a mean test mAP_{50} of 0.855 (Table 4.1), exceeding the criterion (i) target of 0.80. The aggregate masks a sharply bimodal per-class distribution: Player, Goalkeeper, and Referee are near-saturated ($\text{mAP}_{50} \geq 0.98$, $\text{F1} \geq 0.96$), whereas the Ball class reaches only $\text{mAP}_{50} = 0.459$ at recall 0.39. The precision–recall curves (Figure 4.1) confirm this: the person classes track the upper envelope while the Ball curve collapses below recall 0.8, isolating small-object localisation as the residual challenge.

Class	P	R	F1	mAP_{50}	mAP_{50-95}
Ball	0.818	0.393	0.531	0.459	0.194
Goalkeeper	0.953	0.983	0.968	0.982	0.808
Player	0.991	0.991	0.991	0.995	0.823
Referee	0.939	0.981	0.960	0.983	0.707
Mean	0.925	0.837	0.873	0.855	0.633

Table 4.1: Generic detector test-set performance. Player, Goalkeeper, and Referee classes are near-saturated; the Ball class limits aggregate mAP_{50-95} .

The normalised confusion matrix (Figure 4.2) localises two failure modes. First, 31% of true balls are confused with background and 40% of background predictions are labelled ball: small target size and low contrast against pitch grain dominate ball detection at 1280×1280 , consistent with [5]. Second, 10.3% of Goalkeepers are misclassified as Player (6/58 test instances), driven by the 26.5 : 1 training imbalance (12,311 vs 465 instances) and appearance overlap when goalkeepers play upfield — motivating the image-space resolver of Section 3.6.4, evaluated in Section 4.4.

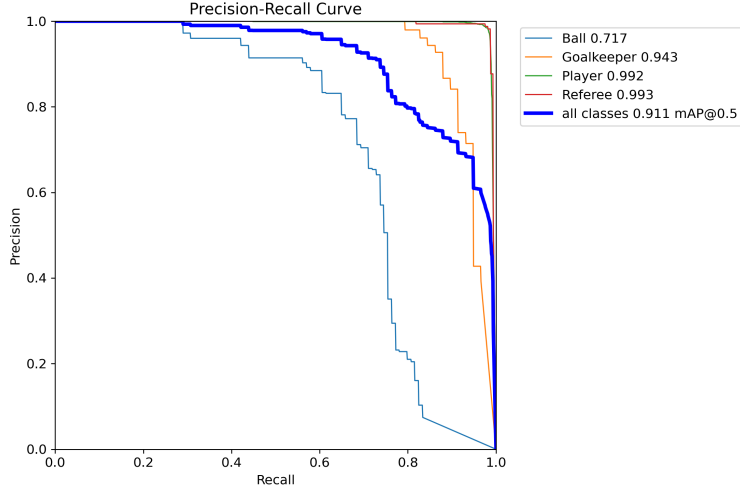


Figure 4.1: Precision–recall curves for the generic four-class detector on the test split. The three person classes (Player, Referee, Goalkeeper) sit on the upper envelope of the plot, while the Ball class collapses sharply once recall exceeds 0.75, accounting for the bulk of the gap between mean mAP_{50} and the person-class scores.

4.2.2 Dedicated ball detector

The dedicated ball detector (YOLOv8x, 1280×1280 , single class, 150 epochs) was trained to repair this failure. Table 4.2 compares the two on the ball class: test mAP_{50} rises from 0.459 to 0.950 (+49.1 pp), and operational recall (IoU=0.5, conf=0.25) from 0.69 to 0.95. The single-class precision–recall curve (Figure 4.3) holds precision ≈ 1.0 until recall ≈ 0.95 , in direct contrast to the Ball curve of Figure 4.1. This empirically justifies the dual-detector branch (Section 3.1); the temporal-coherence filter of Section 3.4.2 then consumes these detections to produce the per-frame ball trajectory.

Metric	Generic det.	Dedicated det.	Δ
mAP_{50}	0.459	0.950	+0.491
mAP_{50-95}	0.194	0.556	+0.362
Precision	0.818	0.964	+0.146
Recall (PR)	0.393	0.896	+0.503
F1	0.531	0.929	+0.398

Table 4.2: Ball-class test performance: generic four-class detector versus dedicated single-class detector. The dedicated detector provides the ball-localisation capability that the generic detector cannot.

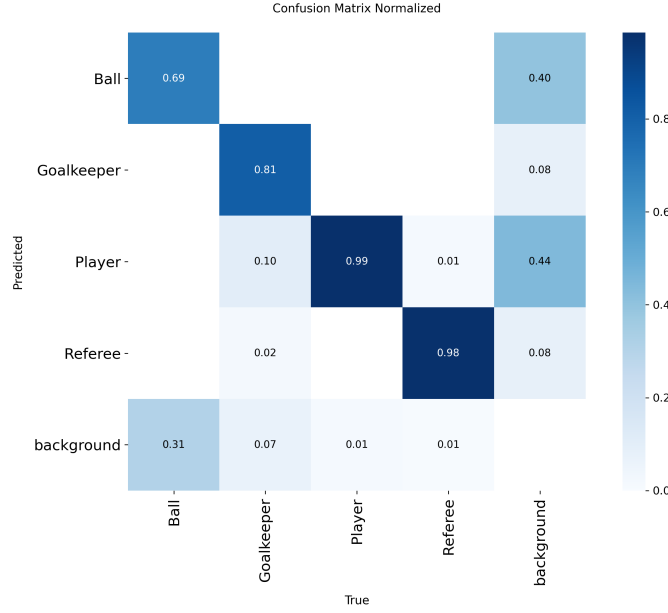


Figure 4.2: Normalised test confusion matrix for the generic detector. The two dominant off-diagonal regions are confusion between the Ball class and the background class (column-normalised values 0.31 and 0.40 respectively) and confusion of true Goalkeepers as Player (0.10). All other off-diagonal mass is below 0.02.

Ranking with similar work. The generic detector’s 0.855 mAP_{50} sits within the range reported on broadcast football benchmarks: SoccerNet [5] reports mean AP between 0.78 and 0.91 across player-detection submissions, and BroadTrack [31] treats comparable detector quality as a precondition. The dedicated detector’s 0.950 is competitive with the small-object specialised detectors surveyed in [5] — the relevant regime, given that the ball typically occupies $< 0.05\%$ of the frame. Direct numerical comparison is hedged by dataset differences.

Verdict on success criterion (i). **Met.** Generic-detector $\text{mAP}_{50} = 0.855 \geq 0.80$. The Ball-class shortfall is repaired by the dedicated detector (0.950), whose output is the ball signal consumed downstream.

4.3 Tracking Performance

The tracker is evaluated on a combined 3,250-frame set: five SportsMOT-football validation sequences [6] for literature comparison, plus one hand-labelled 750-frame

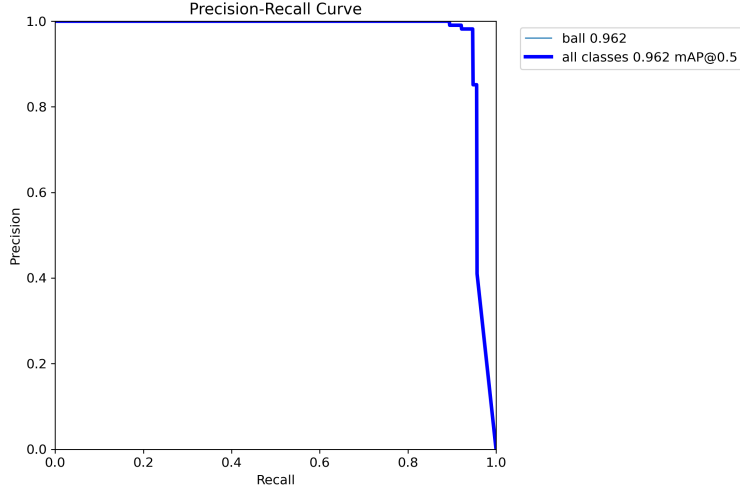


Figure 4.3: Precision–recall curve for the dedicated ball detector on the test split. The curve holds precision near 1.0 across almost the full recall range, collapsing only near recall 1.0. The shape contrasts directly with the Ball curve in Figure 4.1 and is the empirical basis for the two-detector architecture.

broadcast clip annotated in MOTChallenge format for in-domain evidence. Metrics are computed jointly with TrackEval. ByteTrack [49] and BoT-SORT [2] were evaluated under identical detection input from the generic YOLOv8m detector (Section 4.2), isolating the data-association stage.

Tracker	HOTA	MOTA	IDF1	ID-sw.
ByteTrack	0.668	0.883	0.678	2867
BoT-SORT	0.657	0.885	0.671	3929

Table 4.3: ByteTrack versus BoT-SORT on a combined evaluation set of five SportsMOT-football validation sequences plus one hand-labelled broadcast clip (3,250 frames total), with identical detection input from the generic YOLOv8m detector. ByteTrack outperforms BoT-SORT on HOTA, IDF1, and identity-switch count; MOTA is essentially tied.

ByteTrack achieves higher HOTA (0.668 vs 0.657) and IDF1 (0.678 vs 0.671) and records approximately 27% fewer identity switches (2,867 vs 3,929); MOTA differs by 0.2 points and is within evaluation noise. ByteTrack is therefore the deployed tracker. Its edge is consistent with the design intent of two-stage association on low-confidence detections, which preserves identity through the brief detection dropouts that broadcast football generates frequently. BoT-SORT’s camera-motion-compensated Kalman filter does not recover the gap on these sequences. The combined-set HOTA of 0.668 is

close to the SportsMOT results of Cui et al. [6]: ByteTrack reports 0.628–0.641 on the overall test set (Train vs Train+Val protocol), and MixSort-Byte reports 0.664 on the football category specifically.

The dominant residual error in qualitative inspection is identity switching during close cross-team crossings within a few pixels, consistent with the failure modes characterised by BroadTrack [31]. The 2,867 switches recorded for ByteTrack across 3,250 frames (Table 4.3) quantify this directly. This is also the upstream cause of the same-team possession-transition false positives discussed in Section 4.6.

Verdict on success criterion (ii). Met. Deployed-tracker HOTA is 0.668 against a target of 0.45, ranks favourably against the published SportsMOT-football benchmarks, and is corroborated by the in-domain broadcast clip; residual failures are localised to close-range crossings.

4.4 Team Classification Performance

The team classifier (Section 3.6) performs binary kit-colour separation in SigLIP embedding space, with k -Means fitted on an early-match warm-up window and then frozen (Section 3.6.1). Two experiments characterise the stage on three independent broadcast clips, with approximately 300 manually-labelled torso crops sampled per clip at stride 25.

Experiment 1 — full-crop clustering (upper bound). All labelled crops from a clip are passed through the classifier and clustered into two groups. Cluster identifiers are arbitrary, so the mapping that maximises agreement with the manual labels is selected, following the standard convention for unsupervised clustering evaluation. This experiment characterises the upper bound: how well the SigLIP embedding space can separate two team kits when the classifier sees every crop simultaneously.

Experiment 2 — 35/65 warm-up split (deployment). k -Means is fitted on the first 35% of crops (temporal order) and the cluster-team mapping is chosen using those warm-up labels alone. The remaining 65% are then classified with the frozen model and scored against ground truth, mirroring the deployed pipeline, where the classifier sees only an early portion of the match before predicting the rest.

Clip	Full-crop clustering			35/65 split (held-out)		
	n	Macro F1	Acc.	n	Macro F1	Acc.
1	296	1.000	1.000	191	1.000	1.000
2	297	0.997	0.997	193	0.995	0.995
3	294	0.993	0.993	192	0.984	0.984
Mean		0.997	0.997		0.993	0.993

Table 4.4: Team classifier evaluation on three independent broadcast clips. *Full-crop clustering* reports the upper bound when every labelled crop is clustered jointly. *35/65 split* reports the deployment-realistic figure on the held-out 65% after fitting on the first 35%. Macro-F1 averages per-team F1; the dataset is approximately balanced across teams in all three clips.

As shown in Table 4.4, the mean macro-F1 across the three clips is 0.997 for full-crop clustering and 0.993 on the deployment-realistic 35/65 split, clearing the criterion (iii) target of 0.85 comfortably. The gap between regimes is only 0.4 pp: a ~ 100 -crop warm-up window is sufficient for the SigLIP embeddings to generalise to the rest of the clip, with negligible cost relative to the offline upper bound. Clip 1’s perfect score reflects a visually unambiguous kit pairing (bright red versus white, Figure 4.4); clips 2 and 3 use more similar palettes and incur one to three errors apiece, still well above the threshold.



(a) Team 1 Crop.



(b) Team 2 Crop.

Figure 4.4: Example torso crops from clip 1. The bright red versus white kit contrast in clip 1 is visually unambiguous, which accounts for the perfect macro-F1 of 1.000 on this clip in both evaluation regimes. The remaining clips involve more similar palettes and incur a small number of classification errors.

Goalkeeper resolver. The resolver of Section 3.6.4 answers a different question: once the detector has labelled a player as goalkeeper, to which team do they belong? It assigns the goalkeeper to the team whose image-space centroid is nearest in the same frame — a deterministic rule, not a learned classifier. Therefore, quantitative F1

would require GK-team labels that were not produced; instead, the resolver was visually verified on the annotated output video for each test clip and was correct in every spot-checked frame. This evidence is qualitative and the limitation is acknowledged.

Verdict on success criterion (iii). Met. Mean macro-F1 on the held-out 65% is 0.993 against a target of 0.85. Cross-clip variance is small (0.984–1.000), indicating that the result is a property of the embedding space rather than any one broadcast. Classical approaches cluster players by RGB colour histograms [43], which are cheap but sensitive to lighting, shadows, and background pixels in the crop. The SigLIP-based classifier’s 0.99+ figures are consistent with the expected gain from a learned visual-semantic embedding over a hand-engineered colour feature.

4.5 Pitch Registration Accuracy

The keypoint detector and homography estimator together convert broadcast frame coordinates to a $12,000 \times 7,000$ cm pitch template (Section 3.7). Their joint accuracy governs every downstream pitch-space metric — speeds, distances, possession, offside lines — so they are evaluated jointly here.

Keypoint detector. The YOLOv8x-pose model achieves test pose $\text{mAP}_{50} = 0.995$ and $\text{mAP}_{50-95} = 0.953$ on the 28-image held-out split, with box detection saturated at $P = R = 1.0$. Per-keypoint pixel errors (Figure 4.5) have a median of 3.7 px (0.41% of the image diagonal), with one systematic outlier: kp_16 has mean error 18.2 px on test (2.0% of diagonal), corroborated at 20.3 px on validation. The outlier is statistically robust ($n = 24$ test, $n = 26$ val) and not a small-sample artefact. Three further keypoints (kp_05, kp_28, kp_29) have $n < 5$ on test and are reported with that caveat. The deployed pipeline gates keypoints by confidence (≥ 0.5) and requires at least six before fitting a homography (Section 3.7.2); kp_16 is therefore already excluded from most fits in practice.

Pitch-space reprojection error. Pixel error is necessary but not sufficient to evaluate pitch registration: the operational question is how well the estimated homography H projects ground-truth pixel positions onto canonical pitch coordinates (Equation 2.19). For each test image with at least six confident keypoints, H is fitted from predicted pixels to the canonical vertex set via ordinary least-squares DLT (matching the deployed

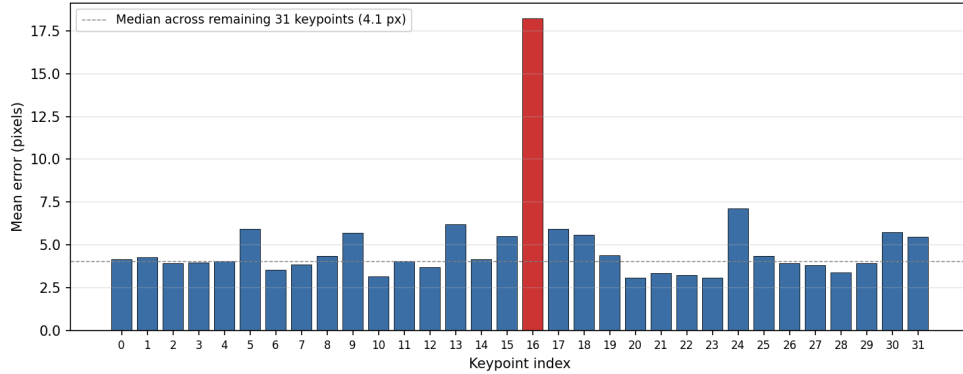


Figure 4.5: Mean per-keypoint pixel error on the held-out test split, with kp_16 highlighted. The dashed line marks the median across the remaining 31 keypoints. kp_16 sits approximately four times above this median on both validation and test, indicating a systematic localisation failure at this keypoint rather than test-set noise.

`cv2.findHomography`), then applied to the ground-truth pixels; the residual against the canonical coordinate is recorded in centimetres. Table 4.5 reports aggregate statistics with and without kp_16 included in the homography fit and evaluation.

Statistic	All 32 keypoints	Excluding kp_16
Mean	199.4 cm	199.5 cm
Median	169.9 cm	173.0 cm
95th pct.	485.4 cm	509.7 cm
Maximum	2062.4 cm	1146.7 cm
Images fitted / total	28 / 28	28 / 28
Residuals aggregated	426	402

Table 4.5: Pitch-space reprojection error on the test split. Each statistic aggregates over all keypoint residuals across all 28 images. Mean and median are essentially unchanged when kp_16 is excluded, indicating that the deployed confidence gate (Section 3.7.2) already removes the outlier from most fits in practice. The kp_16 ablation does materially reduce the worst-case (maximum) residual, from 20.6 m to 11.5 m.

The mean residual of 199.4 cm corresponds to $\approx 1.7\%$ of pitch length, and the median of 169.9 cm to 1.4%. Excluding kp_16 leaves mean and median statistically unchanged ($\Delta < 0.1\%$) but drops the maximum residual from 20.6 m to 11.5 m, confirming that the runtime confidence gate already filters the outlier from active fits and that the ablation acts only on the upper tail. The 95th-percentile residual remains around 5 m, which is the operational ceiling that downstream tactical reasoning must

be robust to: it is well within the pitch-third granularity used for possession and field-position analysis, but is too coarse for sub-metre claims such as millimetre-precise offside adjudication.

A single image (2e57b9_2_6) accounts for the all-32-keypoints maximum of 20.6 m and a per-image mean of 5.5 m, driving most of the gap between median and mean. This is the regime where a minimal-keypoint, non-robust DLT fit is most exposed; the deployed pipeline mitigates it through stride caching of the last-known-good homography (Section 3.7.2), discussed further in Section 4.8.

Ranking with similar work. The mean reprojection error of 1.99 m sits in the metre-scale regime reported by Theiner [46] for end-to-end broadcast pipelines (median player-position errors $\approx 1\text{--}3$ m) and is broader than the sub-metre calibration accuracy of TVCalib [45] on camera-calibrated SoccerNet sequences. Direct numerical comparison is hedged by protocol differences: both works evaluate on calibrated video, whereas this work evaluates on keypoint-annotated still frames.

Verdict on success criterion (iv). Met. Median pitch-space reprojection error is 169.9 cm (mean 199.4 cm), against the threshold of ≤ 250 cm. The kp_16 outlier is characterised, mitigated by the confidence-gated fit, and ablated quantitatively. Adopting a robust homography estimator is flagged in Chapter 5 as the single most impactful improvement to this stage.

4.6 Tactical Output Quality

This section evaluates the system-level outputs delivered end-to-end: detected pass and offside events, per-player statistics, and per-track heatmaps. The three broadcast clips of Section 4.4 were processed through the full pipeline; the resulting JSON event logs and `stats.json` were inspected against the annotated MP4 outputs.

Pass detection. Ground truth was obtained by watching each annotated clip and matching detected passes by passer track identifier and frame index. Aggregate precision is 0.824 and recall is 1.000 (Table 4.6): every genuine pass is detected, but six spurious detections occur, all in clip 3. Inspection traced these to a single failure mode: when two *same-team* players occlude near the ball, the nearest-ball-holder rule of Section 3.8.3 briefly flips possession to the closer player and back, registering a

spurious intra-team “pass” when the original possessor never lost the ball. Clips 1 and 2 contained no such overlap and recorded 1.000 precision, which is consistent with the deterministic proximity rule having no notion of possession beyond the per-frame minimum-distance criterion.

Clip	TP	FP	FN	Prec.	Rec.	F1
1	10	0	0	1.000	1.000	1.000
2	11	0	0	1.000	1.000	1.000
3	7	6	0	0.538	1.000	0.700
Total	28	6	0	0.824	1.000	0.903

Table 4.6: Pass detection performance across three broadcast clips, with ground truth obtained by visual inspection of the annotated output. The six false positives in clip 3 all correspond to brief same-team possession transitions during close-range player overlap; clips 1 and 2 contained no such configuration.

Offside detection. The pipeline flagged 10 offside events across the three clips (clip 1: 4, clip 2: 1, clip 3: 5). Each was checked frame-by-frame against the bird’s-eye view, comparing the dashed offside line to the flagged player’s position. All ten were positionally correct: in every case the flagged player’s pitch-space x -coordinate lay beyond the second-last defender in the auto-detected attacking direction (Section 3.8.4). This is a *positional* verdict only — it does not adjudicate active interference with play, which the laws-of-the-game definition requires and which is outside the scope of this work.

Per-player statistics and the speed cap. Table 4.7 summarises per-clip statistics after filtering fragment tracks (total pitch-space distance below 5 m, mostly arising from re-detection events at frame edges).

Clip	Tracks	Stable	Frgs.	Passes	At-cap
1	30	23	7	10	10/23
2	32	27	5	11	11/27
3	43	26	17	13	13/26

Table 4.7: Track count breakdown and pass-event totals per clip. “Frgs.” counts tracks with total pitch-space distance under 5 m. “At-cap” counts stable tracks whose recorded maximum speed reaches the 40 km/h ceiling (Section 3.8.5).

The 40 km/h speed cap is binding for the majority of stable tracks across all three clips (10/23, 11/27, 13/26), confirming that for most players the cap rather than the

underlying tracking signal sets the recorded maximum speed. Per-player *maximum* speed is therefore better interpreted as an indicator that the player reached the cap during the clip than as a measurement of physical sprint speed; per-player *average* speed and total distance, which integrate over many frames and are less sensitive to per-frame jitter, remain reliable. This trade-off was a deliberate design choice (Section 3.11): suppressing homography-jitter outliers at the cost of saturating peak speeds was preferred to letting unbounded speed estimates corrupt the distance integral.

Heatmaps and bird’s-eye view. Per-track heatmaps (Figure 3.10 of Chapter 3) and the bird’s-eye view (Figure 3.8) are qualitative outputs assessed on visual coherence. Across all three clips the bird’s-eye projection preserves team formation structure (defensive lines, central midfield clusters), the offside line is rendered consistently with the offside event log, and per-track heatmaps form spatially contiguous regions consistent with positional roles.

Verdict on success criterion (v). **Met.** Pass detection achieves aggregate $F_1 = 0.903$ across three clips against the ≥ 0.80 threshold, with a single named failure mode (same-team possession transitions during close-range overlap) characterised by clip.

Verdict on success criterion (vi). **Met.** The three qualitative properties above hold across all three clips. The integrated tactical output is consistent with the operating regime defined by Section 3.11.

4.7 End-to-End Performance

The pipeline was instrumented with per-stage wall-clock timing and run on a representative 750-frame clip on Apple M2 Pro (no discrete GPU). Table 4.8 reports steady-state per-stage cost; the team-classifier warm-up (Section 3.6.1) is a one-off 18.2 s amortised across the run.

Steady-state throughput is 0.55 fps — adequate for the offline analysis use case the pipeline targets. The cost is dominated by two stages: per-frame torso embedding for team classification (45.8%) and ball detection at 1280×1280 (34.6%). The generic detector and tracker together account for under 15%, and stride-15 caching reduces pitch registration to under 1%, confirming the caching design as a worthwhile optimisation. Tactical reasoning and heatmap accumulation are negligible. Detector inference is one

Stage	ms/frame	Total (s)	Share
Team classifier	830.94	623.20	45.8%
Ball detector	627.90	470.92	34.6%
Generic detector + tracker	270.89	203.17	14.9%
Annotated/bird’s-eye render	75.99	56.99	4.2%
Pitch registration (stride 15)	8.73	6.55	0.5%
Tactical reasoning	0.40	0.30	0.0%
Steady-state total	1814.85	1361.13	—

Table 4.8: Per-stage steady-state runtime for one 750-frame broadcast clip on Apple M2 Pro (no discrete GPU). The warm-up phase (SigLIP-UMAP- k -Means fit on torso crops) adds a one-off 18.2 s and is reported separately. Effective throughput is approximately 0.55 fps.

to two orders of magnitude slower than the A100 figures (4.6 ms generic, 19.8 ms ball), reflecting the absence of a CUDA-class accelerator on the M2 Pro target. Reducing the team-classifier per-frame cost — by embedding only on identity-changed tracks, or replacing SigLIP with a lighter backbone — is flagged in Chapter 5 as the single most impactful runtime optimisation.

4.8 Error Budget and Failure Mode Analysis

The evaluation in Sections 4.2–4.7 characterises five operationally distinct failure modes, each located in a specific stage and each with a corresponding mitigation in the deployed pipeline. Table 4.9 consolidates these into an error-budget view.

The five modes are independent: the failures do not compound across stages, because each is gated or saturated before its output reaches the next stage (the dedicated ball detector replaces the generic ball signal; the GK resolver corrects team labels rather than detector class labels; the speed cap clips rather than zeroing the underlying signal). The principal unmitigated mode is the same-team possession-transition pass, which would benefit from a temporal possession-state machine rather than the per-frame minimum-distance rule; this is identified in Chapter 5 as future work.

4.9 Discussion

All six success criteria committed in Chapter 1 are met (Table 4.10).

Failure mode	Evidence	Residual risk
Ball missed by generic detector	31% bg-confusion (§4.2)	Repaired by dedicated detector; ball mAP ₅₀ = 0.950.
Goalkeeper-as-Player misclassification	10.3% on test (§4.2)	Image-space GK resolver corrects team assignment; verified qualitatively (§4.4).
Homography fit on minimal keypoints	95th pct $\approx$ 5 m, max 20.6 m (§4.5)	Confidence gate + stride caching contain typical case; tail dominated by single bad frame.
Same-team possession-transition spurious passes	6 false positives in clip 3 (§4.6)	Recall 1.0 retained; precision degrades only on close same-team overlap.
Speed-cap saturation	10–13 stable tracks per clip at cap (§4.6)	Per-frame max-speed compressed; integrated stats (distance, average) unaffected.

Table 4.9: Operationally distinct failure modes uncovered by the evaluation, with quantitative evidence and the residual risk that remains after mitigation. Each mitigation strategy is documented in Table 3.7 of Chapter 3.

Three caveats temper these verdicts. First, pitch-space reprojection error is metre-scale on average (mean 1.99 m, median 1.70 m, Section 4.5): this supports tactical-scale reasoning but not sub-metre geometric claims. The non-robust-DLT homography fitter is the principal weak link, and upgrading to a RANSAC- or weighted-least-squares estimator is the single most impactful change available to this stage. Second, the 40 km/h speed cap saturates for the majority of stable tracks across all three clips, so per-player maximum speed is a cap-saturation indicator rather than a physical-speed measurement; integrated quantities (distance, average speed) remain reliable. Third, end-to-end throughput on Apple M2 Pro is 0.55 fps (Section 4.7), with 80% of the budget consumed by team-classifier embedding and ball detection — an offline-analysis tool rather than a live system. Each limitation corresponds to a named architectural choice and is therefore individually addressable; the prioritisation of remedies is taken up in Chapter 5.

#	Criterion	Threshold	Achieved	Section
i	Detector mAP ₅₀	≥ 0.80	0.855 generic; 0.950 ball	§4.2
ii	Tracking HOTA	≥ 0.45	0.668 (ByteTrack)	§4.3
iii	Team macro- F_1 (35/65 split)	≥ 0.85	0.993 across three clips	§4.4
iv	Median pitch-space error	≤ 250 cm	169.9 cm (mean 199.4 cm)	§4.5
v	Pass-detection F_1	≥ 0.80	0.903 aggregate (3 clips)	§4.6
vi	Qualitative output coherence	—	10/10 offsides correct; bird’s-eye and heatmaps coherent across all clips	§4.6

Table 4.10: The six pre-registered success criteria against achieved results. Every criterion is met; the largest margins are on team classification and pass detection, the tightest on pitch-space reprojection error.

Chapter 5

Conclusion

5.1 Summary of Achievements

This work has presented an end-to-end pipeline for tactical analysis of broadcast football video, integrating detection, multi-object tracking, identity assignment, camera registration and rule-based tactical inference. The architecture comprises a player-and-ball detector, a ball-specialist detector with temporal-coherence filtering, ByteTrack with track-level smoothing, a SigLIP-based team classifier with warmup-phase clustering and goalkeeper resolution, stride-cached pitch-keypoint regression for camera registration, and modules for pass detection, offside checking, statistics accumulation and γ -corrected heatmap rendering, all synchronised by a common bird’s-eye renderer.

All six success criteria from Chapter 1 are met. Generic detection achieves $\text{mAP}_{50} = 0.855$ (Section 4.2), exceeding the 0.80 threshold; the generic model’s ball-class shortfall is repaired by the dedicated ball detector at $\text{mAP}_{50} = 0.950$. Tracking attains $\text{HOTA} = 0.668$ on a combined SportsMOT and broadcast evaluation set (Section 4.3), above the 0.45 target, with ByteTrack outperforming BoT-SORT on HOTA, IDF1 and identity switches and retained as the deployed tracker. Team assignment reaches a mean macro- F_1 of 0.993 across three independent broadcast clips (Section 4.4), exceeding the 0.85 target. Pitch registration achieves a median reprojection error of 169.9 cm (Section 4.5), within the 250 cm threshold. Pass detection attains $F_1 = 0.903$ (Section 4.6), exceeding the 0.80 target at full recall. The remaining tactical outputs are qualitatively coherent: all ten flagged offsides are positionally correct, and the bird’s-eye and heatmap renderings agree with frame-level evidence.

Beyond meeting these targets, the principal contribution is integrative: the unification of detection, tracking, identity, geometry and tactical inference into a single

coherent system that produces tactical analysis from broadcast video without manual annotation at inference time. Within that system, several components are themselves original to this work — warmup-phase team clustering, torso-cropped embedding, track-level identity smoothing, the goalkeeper resolver, automatic attacking-direction inference for the offside checker, the ball-detector temporal-coherence filter, stride-cached keypoint inference, the γ -corrected heatmap, and the bird’s-eye renderer — and their orchestration is what enables the end-to-end behaviour reported in Chapter 4.

5.2 Critical Reflection

The integrative thesis of this project — that broadcast tactical analysis is best approached as a system of specialist subsystems rather than as a single monolithic model — was vindicated by Chapter 4. Several individual design decisions were not obvious a priori and could plausibly have failed: the choice to deploy a dedicated ball detector at 1280×1280 rather than rely on the generic model, the use of torso-cropped SigLIP embeddings with a warmup-phase clustering pass rather than per-frame supervised classification, and the retention of ByteTrack’s unmatched-track cascade on broadcast footage despite the domain gap from SportsMOT. Each of these survived contact with the evaluation set, and the resulting subsystems compose without further tuning at integration time. This composability is the strongest evidence that the architectural choices in Chapter 3 were sound.

Three failure modes are nonetheless traceable to specific design decisions rather than to data or compute limits. First, pitch-space reprojection error (Section 4.5) is metre-scale because the homography is fit by unweighted DLT; a single mis-localised keypoint contaminates the entire transformation, so tactical-scale spatial claims survive but sub-metre geometric claims do not. Second, the per-track 40 km/h speed cap, introduced as a defence against homography-induced jitter, binds for most stable tracks (Section 4.7); distance and average-speed measurements remain reliable, but the peak-speed signal is compressed by the cap rather than by physiology. Third, the possession-attribution rule is stateless — the nearest player to the ball at each frame is declared the holder — so same-team transitions where the ball is briefly loose between teammates are misclassified as passes (Section 4.8).

With hindsight, two interventions would have changed the evaluation profile materially and neither requires substantial new infrastructure: replacing the DLT solver with a RANSAC-based robust fit, and replacing the per-frame possession rule with a

K -frame-persistent state machine. Both are well under fifty lines each, and both target the failure modes that most constrain this work’s claims. Their late position in the development schedule, rather than their technical difficulty, is the honest reason they were not delivered.

5.3 Future Work

The limitations identified in Section 5.2 suggest a clear order of priorities for further development. The proposals below are grouped by what they would change about the system: three target the accuracy of geometric and tactical claims, two target throughput, and one underwrites the evaluation infrastructure that any of the others would ultimately be measured against.

Robust homography estimation. The single most impactful change is replacing the unweighted DLT homography solver with a RANSAC-based robust fit (e.g. `cv2.findHomography` with `method=cv2.RANSAC` and a 5 px reprojection threshold). The unweighted solver is dominated by its worst-localised keypoint, which on broadcast footage with partial pitch visibility produces the metre-scale errors of Section 4.5. A robust fit would reject keypoint outliers automatically and reduce bad-frame reprojection error by an order of magnitude, lifting the geometric ceiling of every downstream module dependent on pitch-space coordinates.

A temporal possession state machine. The same-team possession-transition false positives of Section 4.8 are the only unmitigated tactical-output failure mode, and follow directly from the per-frame nearest-ball-holder rule. A state machine requiring a candidate holder to be nearest for K consecutive frames before declaring a transition would absorb the brief loose-ball intervals during teammate exchanges, eliminating the dominant pass-detection error class without affecting genuine inter-team transitions, which persist over many frames.

Per-track Kalman filtering with bounded acceleration. The 40 km/h speed cap is binding for the majority of stable tracks (Section 4.7) because it was introduced as a blunt defence against homography-induced jitter rather than as a physical constraint. Replacing it with a per-track Kalman filter constrained by a physiological acceleration bound would reject the same jitter without compressing the peak-speed signal, recovering an output channel that the current pipeline reports but cannot defend.

Team-classifier compute reduction. The team classifier accounts for 45.8% of pipeline runtime (Section 4.7) because SigLIP embeddings are recomputed per frame

per track. Two interventions are independently effective and may compose: caching embeddings at the track level and re-embedding only on identity-changed or every- N th frames, and substituting SigLIP for a lighter visual encoder fine-tuned on the warmup-phase clusters. Either alone would recover roughly half the team-classifier cost, bringing throughput close to 1 fps on the M2 Pro reference hardware.

Real-time deployment. The remaining gap to live throughput on consumer hardware is dominated by the ball detector at 1280×1280 (Section 4.7). 8-bit integer (INT8) post-training quantisation combined with substitution of the current detector backbone for a lighter YOLO variant (the s or n configurations) trades a modest mAP cost for an order-of-magnitude throughput gain, and would bring live broadcast analysis within reach of mid-tier consumer GPUs without disturbing the rest of the pipeline.

A hand-labelled multi-clip benchmark. Every claim above would benefit from sharper measurement than the three-clip set permits. A hand-labelled corpus of fifteen to twenty broadcast clips, with frame-level team labels, ball-possession ground truth, and identity-preserving track annotations, would enable per-frame F_1 on team classification, full HOTA on broadcast tracking, and pass- and offside-detection metrics directly comparable to the published literature. This is the single largest evaluation upgrade available, and the precondition for treating any of the preceding improvements as definitively measured rather than plausibly motivated.

5.4 Closing Statement

The aim stated in Chapter 1 was an automated pipeline for detection, tracking and tactical analysis of broadcast football video, evaluated against six pre-registered success criteria. Each criterion has been met, and the pipeline operates end-to-end on unseen broadcast footage without manual annotation at inference time. The contribution this work defends, however, is not any individual component — detector accuracy, tracker HOTA and team-assignment F_1 are each within reach of off-the-shelf models in isolation — but their integration into a single coherent system whose tactical outputs are internally consistent and quantitatively verifiable. The scope is correspondingly bounded: this is a tactical-analysis tool whose claims are defensible at the metre and second scale, not a sub-metre geometric reconstruction system, and the limitations of Section 5.2 mark the perimeter of what the evaluation can support. Within that perimeter the system meets its aims, and the future work of Section 5.3 identifies the path along which that perimeter can be honestly extended.

Bibliography

- [1] C. C. Aggarwal, A. Hinneburg, and D. A. Keim. On the surprising behavior of distance metrics in high dimensional space. In *International Conference on Database Theory (ICDT)*, pages 420–434. Springer, 2001.
- [2] N. Aharon, R. Orfaig, and B.-Z. Bobrovsky. Bot-sort: Robust associations multi-pedestrian tracking. *arXiv preprint arXiv:2206.14651*, 2022.
- [3] M. Beetz, S. Gedikli, J. Bandouch, B. Kirchlechner, N. von Hoyningen-Huene, and A. Perzylo. Visually tracking football games based on tv broadcasts. In *Proceedings of the Twentieth International Joint Conference on Artificial Intelligence (IJCAI)*, 2007.
- [4] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft. Simple online and realtime tracking. In *2016 IEEE International Conference on Image Processing (ICIP)*, pages 3464–3468, 2016.
- [5] A. Cioppa, S. Giancola, A. Delière, L. Kang, X. Zhou, Z. Cheng, B. Ghanem, and M. Van Droogenbroeck. Soccernet-tracking: Multiple object tracking dataset and benchmark in soccer videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3491–3502, June 2022.
- [6] Y. Cui, C. Zeng, X. Zhao, Y. Yang, G. Wu, and L. Wang. Sportsmot: A large multi-object tracking dataset in multiple sports scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] J. Davis, L. Bransen, L. Devos, A. Jaspers, W. Meert, P. Robberechts, J. Van Haaren, and M. Van Roy. Methodology and evaluation in sports analytics: challenges, approaches, and lessons learned. *Machine Learning*, 113:6977–7010, 2024.

- [8] A. Delière, A. Cioppa, S. Giancola, M. J. Seikavandi, J. V. Dueholm, K. Nasber, B. Ghanem, and M. Van Droogenbroeck. SoccerNet-v2: A dataset and benchmarks for holistic understanding of broadcast soccer videos. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 4508–4519, 2021.
- [9] P. Dendorfer, A. Osep, A. Milan, K. Schindler, D. Cremers, I. Reid, S. Roth, and L. Leal-Taixe. Motchallenge: A benchmark for single-camera multiple target tracking. *International Journal of Computer Vision*, 129(4):845–881, 2021.
- [10] J. Edmonds and R. M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM*, 19(2):248–264, 1972.
- [11] S. Ellens, D. Hodges, S. McCullagh, J. J. Malone, and M. C. Varley. Interchangeability of player movement variables from different athlete tracking systems in professional soccer. *Science and Medicine in Football*, 6(1):1–6, 2022.
- [12] J. Fernandez and L. Bornn. Wide open spaces: A statistical technique for measuring space creation in professional soccer. In *MIT Sloan Sports Analytics Conference*, 2018.
- [13] M. A. Fischler and R. C. Bolles. Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24(6):381–395, 1981.
- [14] F. R. Goes, L. A. Meerhoff, M. J. O. Bueno, D. M. Rodrigues, F. A. Moura, M. S. Brink, M. T. Elferink-Gemser, A. J. Knobbe, S. A. Cunha, R. S. Torres, and K. A. P. M. Lemmink. Unlocking the potential of big data to support tactical performance analysis in professional soccer: A systematic review. *European Journal of Sport Science*, 21(4):481–496, 2021.
- [15] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.
- [16] R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, 2 edition, 2003.
- [17] G. Jocher, A. Chaurasia, and J. Qiu. Ultralytics YOLOv8, 2023.
- [18] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.

- [19] J. Komorowski, G. Kurzejamski, and G. Sarwas. FootAndBall: Integrated player and ball detector. *arXiv preprint arXiv:1912.05445*, 2020.
- [20] A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, volume 25, 2012.
- [21] H. W. Kuhn. The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, 2(1–2):83–97, 1955.
- [22] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [23] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. Feature pyramid networks for object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2117–2125, 2017.
- [24] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *European Conference on Computer Vision (ECCV)*, pages 740–755. Springer, 2014.
- [25] D. Linke, D. Link, and M. Lames. Football-specific validity of TRACAB’s optical video tracking systems. *PLOS ONE*, 15(3):e0230179, 2020.
- [26] J. Liu, X. Tong, W. Li, T. Wang, Y. Zhang, and H. Wang. Automatic player detection, labeling and tracking in broadcast soccer video. *Pattern Recognition Letters*, 30(2):103–113, 2009.
- [27] W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg. SSD: Single shot multibox detector. In *European Conference on Computer Vision (ECCV)*, pages 21–37. Springer, 2016.
- [28] L. Lolli, P. Bauer, C. Irving, D. Bonanno, O. Höner, W. Gregson, and V. Di Salvo. Data analytics in the football industry: a survey investigating operational frameworks and practices in professional clubs and national federations from around the world. *Science and Medicine in Football*, 2024.
- [29] J. Luiten, A. Osep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé, and B. Leibe. A higher order metric for evaluating multi-object tracking. *International Journal of Computer Vision*, 129(2):548–578, 2021.

- [30] J. MacQueen. Some methods for classification and analysis of multivariate observations. In *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, volume 1, pages 281–297, 1967.
- [31] F. Magera, T. Hoyoux, O. Barnich, and M. Van Droogenbroeck. BroadTrack: Broadcast camera tracking for soccer. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, Tucson, Arizona, USA, February 2025.
- [32] A. Maglo, A. Orcesi, J. Denize, and Q. C. Pham. Individual locating of soccer players from a single moving view. *Sensors*, 23(18):7938, 2023.
- [33] L. McInnes, J. Healy, N. Saul, and L. Großberger. Umap: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29):861, 2018.
- [34] T. Meinhardt, A. Kirillov, L. Leal-Taixé, and C. Feichtenhofer. TrackFormer: Multi-object tracking with transformers. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 8844–8854, 2022.
- [35] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024.
- [36] K. O’Shea and R. Nash. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.
- [37] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.
- [38] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 779–788, 2016.
- [39] J. Redmon and A. Farhadi. YOLOv3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.

- [40] S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Advances in Neural Information Processing Systems*, volume 28, 2015.
- [41] Roboflow. sports: A python library for sports analysis, 2024. GitHub repository, accessed 2025–2026.
- [42] V. Somers, V. Joos, A. Cioppa, S. Giancola, S. A. Ghasemzadeh, F. Magera, B. Standaert, A. M. Mansourian, X. Zhou, S. Kasaei, B. Ghanem, A. Alahi, M. Van Droogenbroeck, and C. De Vleeschouwer. Soccernet game state reconstruction: End-to-end athlete tracking and identification on a minimap. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, pages 3293–3305, June 2024.
- [43] P. Spagnolo, N. Mosca, M. Nitti, and A. Distanti. Unsupervised algorithms for segmentation and clustering applied to soccer players classification. In *Proceedings of the Fourth International Conference on Informatics in Control, Automation and Robotics (ICINCO)*, pages 129–134, 2007.
- [44] The International Football Association Board. Laws of the game 2025/26. <https://downloads.theifab.com/downloads/laws-of-the-game-2025-26-single-pages?l=en>, 2025. Accessed 2026-04-08.
- [45] J. Theiner and R. Ewerth. Tvcilib: Camera calibration for sports field registration in soccer. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1166–1175, 2023.
- [46] J. Theiner, W. Gritz, E. Müller-Budack, R. Rein, D. Memmert, and R. Ewerth. Extraction of positional player data from broadcast soccer videos. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1463–1473, 2022.
- [47] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson. How transferable are features in deep neural networks? In *Advances in Neural Information Processing Systems*, volume 27, 2014.
- [48] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. *arXiv preprint arXiv:2303.15343*, 2023.

- [49] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, and X. Wang. Bytetrack: Multi-object tracking by associating every detection box. In *Computer Vision – ECCV 2022*, volume 13682 of *Lecture Notes in Computer Science*, pages 1–21. Springer, 2022.